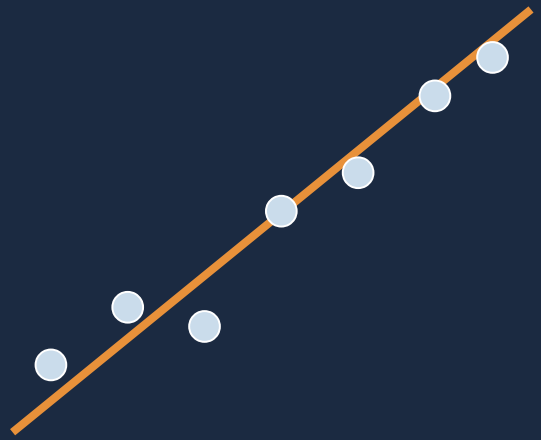
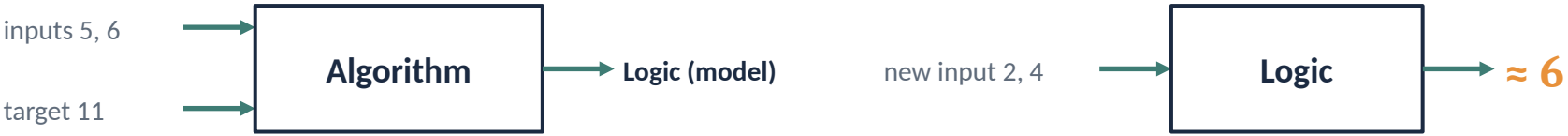
Linear Regression



What is supervised learning?

- Training data is labelled: every example has inputs and a target (label)
- Goal: learn a mapping function that predicts the target for new, unseen inputs
- Example — addition: inputs 5 and 6, target = 11
- Works even for complicated, high-dimensional input data



Training with training data

Predicting with new data

We may not get exactly 6 — a value close to 6, depending on the training data and algorithm.

Two types of supervised learning

Supervised Learning

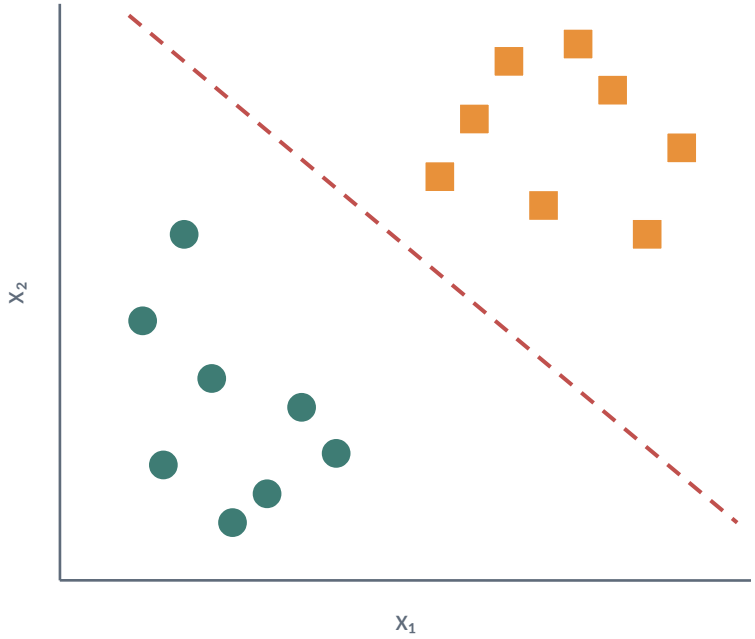
Regression

- Predicts a continuous value (−∞ to +∞)
- Price of a house in a city? Value of a stock?
- Total runs on board in a cricket game?
- Deals with the magnitude of the response

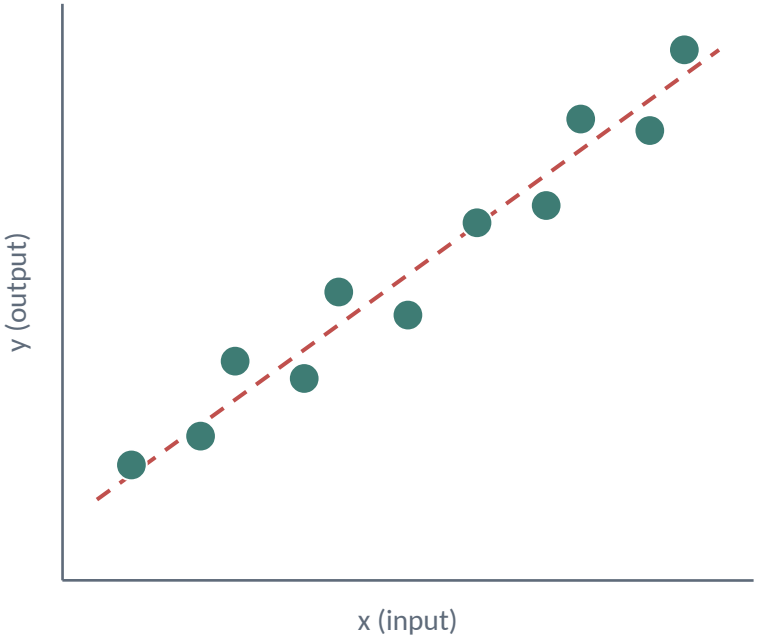
Classification

- Predicts a discrete category (class)
- Binary: spam or not? rain today or not? cat or not?
- Multi-class: spam / important / promotion? cat / dog / tiger?
- Assigns each input to a specific group

Classification separates; regression fits



Classification — find a boundary



Regression — fit a line through the points

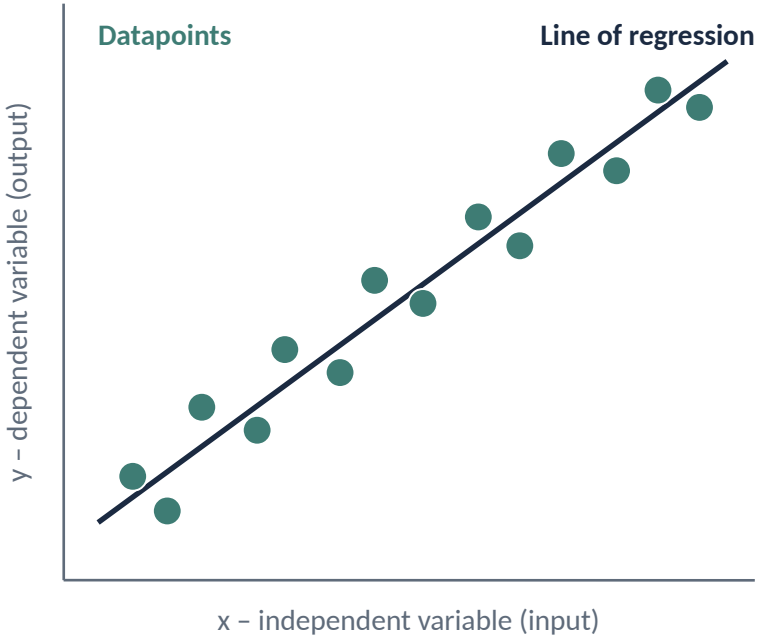
Linear Regression

The relationship between the dependent and independent variable(s) is linear — find the line (or hyperplane) of best fit

Linear regression: the idea

Fit the data with the best line / hyperplane that "goes through" the points

- Models a linear relationship between a dependent variable (y, output) and one or more independent variables (x, inputs)
- Output is continuous → a form of function approximation
- Many lines are possible — we need the one with minimum total error: the line of regression
- House price dataset: Area, Bedrooms, Age, Location = independent variables; Price = dependent variable



Simple linear regression equation

One independent variable x

$$Y_i = \beta_0 + \beta_1 X_i$$

Y_i Dependent variable	β_0 Constant / intercept	β_1 Slope / coefficient	X_i Independent variable

The two parameters we learn (used throughout this lecture):

Parameter	Name	Meaning
β_0	Intercept	Predicted value $\hat{y}$ when x = 0 (where the line crosses the y-axis)
β_1	Slope	Change in $\hat{y}$ for a one-unit increase in x
$\hat{y} = \beta_0 + \beta_1 x$	Prediction	Value on the fitted line for a given x

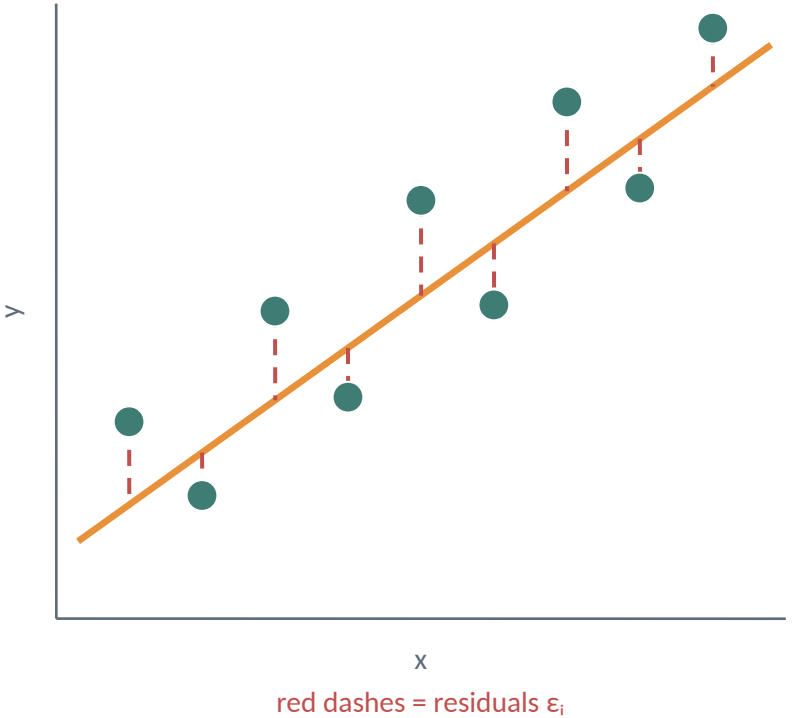
Residual (error) of each point

Every datapoint is approximated by the line; the gap is the residual

$$\varepsilon_i = y_i - \hat{y}_i$$

Symbol	Meaning
ε_i	Error term for a single predicted value
y_i	Actual (observed) value
$\hat{y}_i$	Estimated (predicted) value on the line

Actual = Predicted + Residual
Residuals can be negative, so we square them before adding; the best β_0, β_1 make the total squared error minimum.



red dashes = residuals ε_i

Cost function $J(\beta_0, \beta_1)$

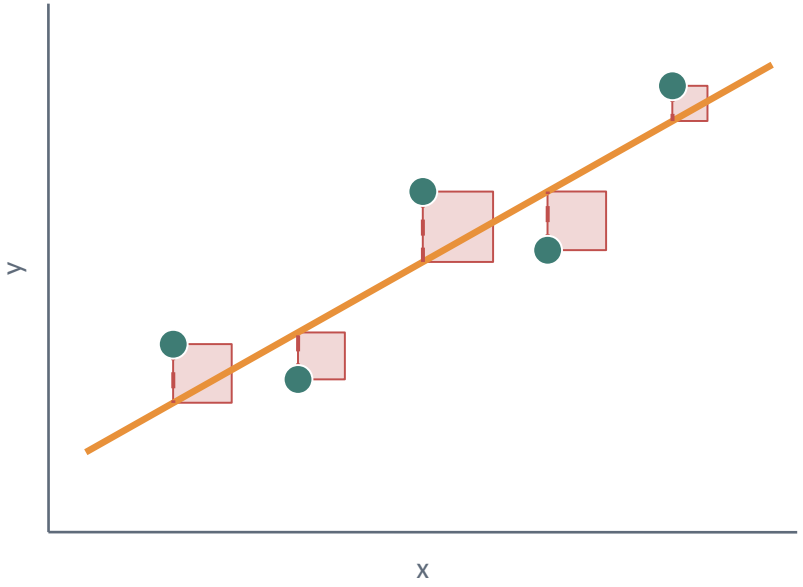
One number that scores how well a line fits all the data

$$J(\beta_0, \beta_1) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

n	y_i	$\hat{y}_i = \beta_0 + \beta_1 x_i$
number of datapoints	actual value	predicted value

Goal: choose the β_0, β_1 that make J as small as possible.

- Square each error (y − ŷ), then add them up for i = 1 to n.
- Squaring makes every error positive and punishes large errors more.
- The line with the lowest cost is the line of regression.



Each red square has area $(y_i - \hat{y}_i)^2$; J adds up all the squares.

Regression Metrics

Evaluating the accuracy of the model

Metric	Good model
SSE (Sum of Squared Errors)	Low (close to 0)
MSE (Mean Squared Error)	Low (close to 0)
RMSE (Root Mean Squared Error)	Low (close to 0)
MAE (Mean Absolute Error)	Low (close to 0)
MAPE (Mean Absolute Percentage Error)	Low (below 10% is very good)
R^2 (Coefficient of Determination)	High (close to 1)

Classification Metrics

Evaluating the accuracy of the model

Metric	Good model
Accuracy	High (close to 1 or 100%)
Precision	High (close to 1)
Recall (Sensitivity)	High (close to 1)
Specificity	High (close to 1)
F1-score	High (close to 1)

Least-squares method: finding β_0 and β_1

For the model $\hat{y} = \beta_0 + \beta_1 x$

1. **Means**

$$\bar{x} = (1/n)\,\Sigma\, x_i \qquad \bar{y} = (1/n)\,\Sigma\, y_i$$

2. **Variance of x**

$$\mathrm{Var}(x) = 1/(n-1)\cdot\Sigma\,(x_i - \bar{x})^2$$

3. **Covariance of x and y**

$$\mathrm{Cov}(x, y) = 1/(n-1)\cdot\Sigma\,(x_i - \bar{x})(y_i - \bar{y})$$

4. **Slope**

$$\beta_1 = \mathrm{Cov}(x, y)\ /\ \mathrm{Var}(x)$$

5. **Intercept**

$$\beta_0 = \bar{y} - \beta_1\,\bar{x}$$

Worked example — Steps 1 to 3

Fit $\hat{y} = \beta_0 + \beta_1 x$ to five points: means, variance and covariance

i	x	y	x − x̄	y − ȳ	(x−x̄)(y−ȳ)	(x−x̄)²
1	1	1.00	−2	−1.06	2.12	4
2	2	2.00	−1	−0.06	0.06	1
3	3	1.30	0	−0.76	0.00	0
4	4	3.75	1	1.69	1.69	1
5	5	2.25	2	0.19	0.38	4
Σ	15	10.30			4.25	10

Step 1 — Means (n = 5)

$$\bar{x} = 15/5 = 3.0 \qquad \bar{y} = 10.30/5 = 2.06$$

Step 2 — Variance of x

$$\mathrm{Var}(x) = 10 \;/\; 4 = 2.5$$

Step 3 — Covariance

$$\mathrm{Cov}(x, y) = 4.25 \;/\; 4 = 1.0625$$

Divide by $n - 1 = 4$ for the sample variance and covariance.

Worked example — Steps 4 and 5

Slope β_1, intercept β_0 and the final model

Step 4 — Slope

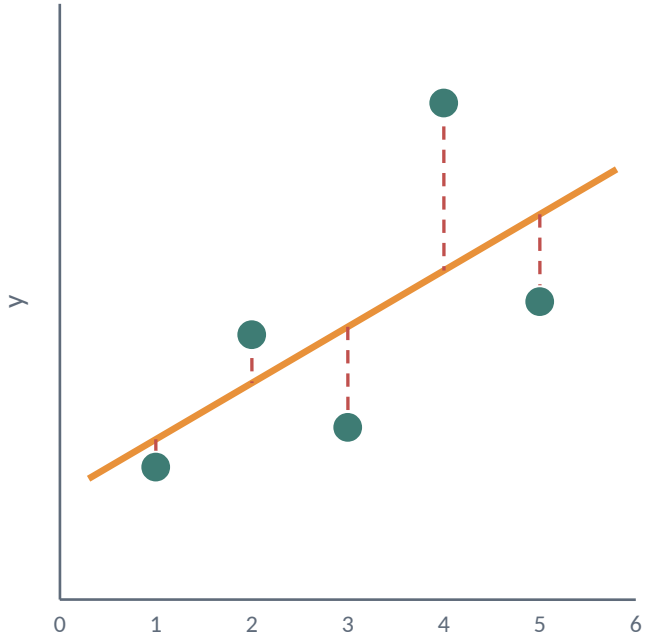
$$\beta_1 = \mathrm{Cov}(x, y) \,/\, \mathrm{Var}(x) = 1.0625 \,/\, 2.5 = 0.425$$

Step 5 — Intercept

$$\beta_0 = \bar{y} - \beta_1 \bar{x} = 2.06 - 0.425 \times 3.0 = 0.785$$

$$\hat{y} = 0.785 + 0.425x$$

Prediction for x = 6: $\hat{y} = 0.785 + 0.425 \times 6 = 3.335$



Multiple linear regression

More than one independent variable

Definition

When the dependent variable y depends linearly on two or more independent variables x_1, x_2, …, x_p, the model is called multiple linear regression. With p features we find β_0, β_1, …, β_p that minimise the squared error; the fitted surface is a hyperplane instead of a line.

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p$$

$\hat{y}$ Predicted output	β_0 Intercept	$\beta_1 \ldots \beta_p$ Coefficient of each feature	$x_1 \ldots x_p$ Independent variables

Assumptions behind linear regression

Linearity

The relationship between a feature and the mean of the dependent variable is linear

y changes with x at a steady rate, so a straight line is the right shape for the data.

Independence

Each random sample (observation) is independent of the others

each observation is separate, and one data point doesn't influence another

Homoscedasticity (Equal variance)

Variance of the residuals is the same for any value of x

The spread of the residuals (errors) is about the same for small and large x values.

Normality

For any fixed value of x, y is normally distributed

for any given x, the y values (equivalently, the residuals) follow a **normal, bell-shaped distribution** around the line. Most points are close to the line and few are far away.

Drawback: if any assumption is violated, the model's results are unreliable — and it is hard for all of them to hold true in real-world data.

When to use linear regression

Continuous outcome; relationship roughly a straight line

Predicting quantity

Forecasting sales, calls to a call centre, electricity consumption

Relationship analysis

Strength and direction of the effect of predictors on the outcome

Homoscedastic data

Residuals roughly equal across all levels of the inputs

Parametric assumptions met

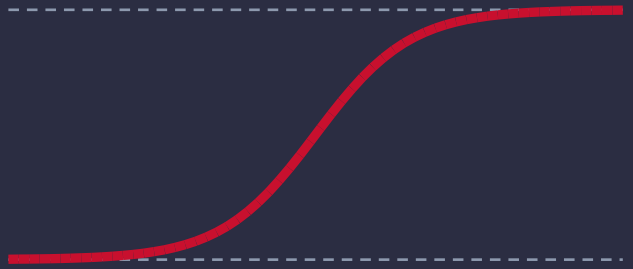
Linearity, independence and normality are satisfied

Use cases

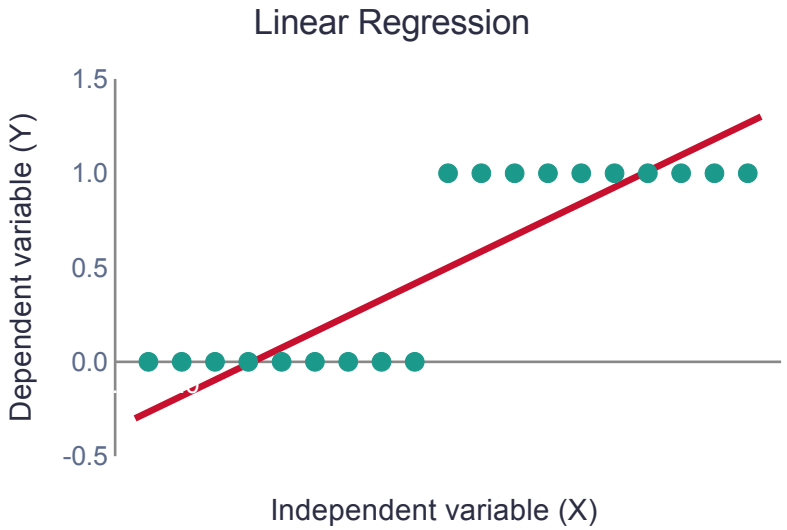
- House price prediction
- Identifying relationships
- Market trends and analysis
- Business risk assessment
- Scientific research
- Price estimation
- Understanding impact

Logistic Regression

Algorithm & Solved Example



Linear vs Logistic Regression





Straight line: predicted Y can exceed the 0 and 1 range

S-curve: predicted Y lies within the 0 and 1 range

Why Logistic Regression?



Linear regression

Predicts the numerical response but is not suitable for predicting the categorical variables.



Classification problem

When categorical variables are involved, it is called a classification problem.



Binary classification

Logistic regression is suitable for binary classification problems.

Predicting Categorical Variables

For example, the following scenarios are instances of predicting categorical variables:



1

Spam or not spam?

Is the mail spam or not spam? The answer is yes or no. Thus, the categorical dependent variable is a binary response of yes or no.



2

Admitted or not?

Whether the student should be admitted or not is based on entrance examination marks. Here, the categorical variable response is admitted or not.



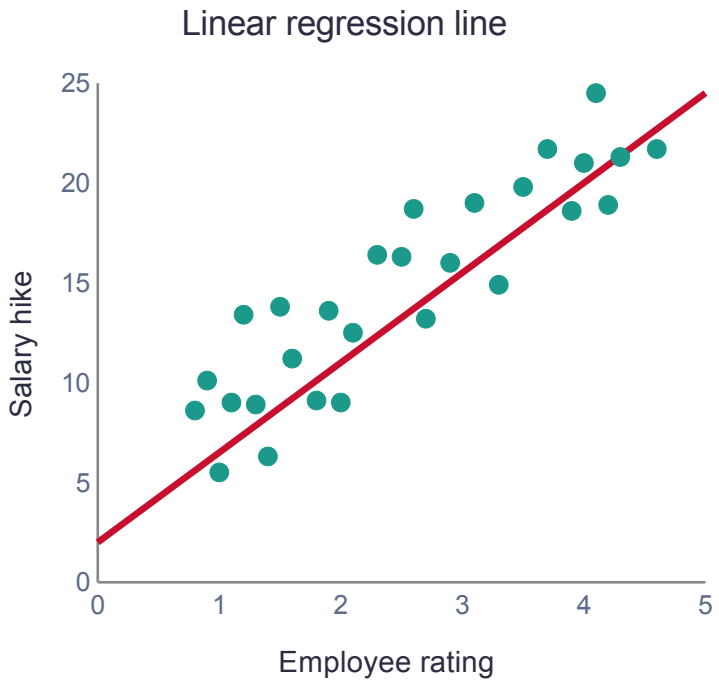
3

Pass or fail?

The student being pass or fail is based on marks secured.

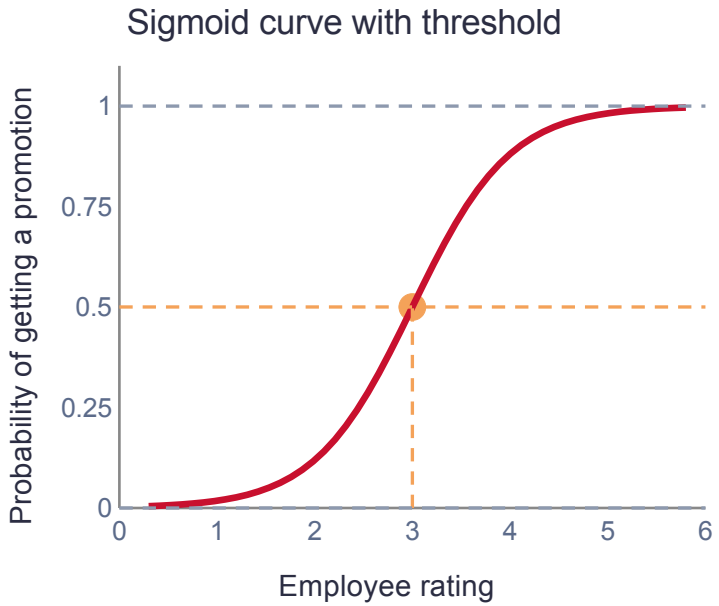
How Does the Algorithm Work?

- Consider the following example:
- An organization wants to determine an employee's salary increase based on their performance.
- For this purpose, a linear regression algorithm will help them decide.
- Plotting a regression line by considering the employee's performance as the independent variable, and the salary increase as the dependent variable will make their task easier.



From a Straight Line to an S-Curve

- Now, what if the organization wants to know whether an employee would get a promotion or not based on their performance?
- The above linear graph won't be suitable in this case.
- As such, we clip the line at zero and one, and convert it into a sigmoid curve (S curve).
- Based on the threshold values, the organization can decide whether an employee will get a salary increase or not.

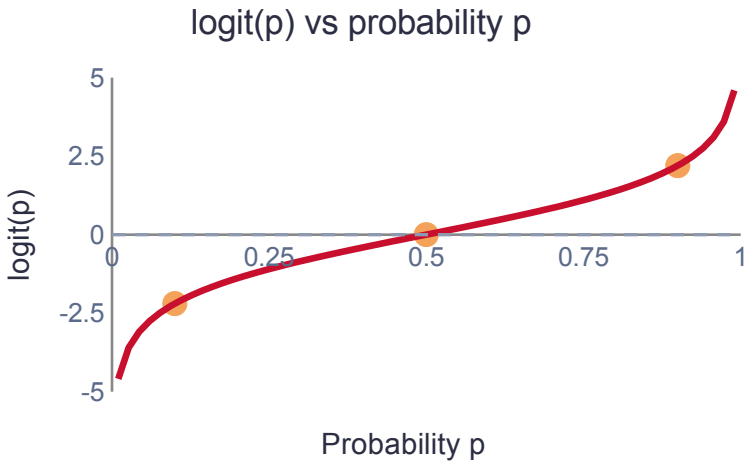


Threshold (th) = 0.5 → cut-off rating = 3

The Logit Function

$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x$$

- The log of the odds is called the logit (log-odds) of p.
- Logit maps a probability between 0 and 1 to any real number from −∞ to +∞.
- The logit is linear in x, which is why logistic regression is a “regression”: we fit a straight line to the log-odds.
- The sigmoid is the inverse of the logit: it maps $\beta_0 + \beta_1 x$ back to a probability.
- β_1 is the change in log-odds for a one-unit increase in x; e^β_1 is the odds ratio.



p	Odds p/(1−p)	logit(p)
0.1	0.11	−2.20
0.5	1	0
0.9	9	+2.20

The Odds of Success

To understand logistic regression, let's go over the odds of success.

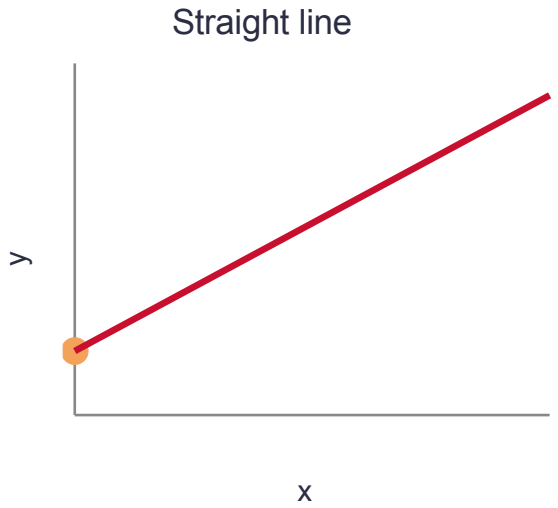
$$\text{Odds } (\theta) = \frac{\text{Probability of an event happening}}{\text{Probability of an event not happening}}$$

$$\text{Odds } (\theta) = \frac{p}{1-p}$$

The values of odds range from **zero to ∞** and the values of probability lie between **zero and one.**

Consider the equation of a straight line:

$$y = \beta_0 + \beta_1 * x$$



β_0 = intercept β_1 = slope

Taking the Log of the Odds

1 Now to predict the odds of success, we take log on the odds formula:

$$\log\left(\frac{p(x)}{1-p(x)}\right)=\beta_0+\beta_1 x$$

2 Exponentiating both the sides, we have:

$$e^{\ln\left(\frac{p(x)}{1-p(x)}\right)}=e^{\beta_0+\beta_1 x}$$

$$\downarrow$$

$$\left(\frac{p(x)}{1-p(x)}\right)=e^{\beta_0+\beta_1 x}$$

Using the identity

$$e^{\ln(x)}=x$$

Deriving the Sigmoid Function

Let $Y = e^{\beta_0 + \beta_1 * x}$

Then $\frac{p(x)}{1 - p(x)} = Y$

$p(x) = Y(1 - p(x))$

$p(x) = Y - Y(p(x))$

$p(x) + Y(p(x)) = Y$

$p(x)(1 + Y) = Y$

$p(x) = \frac{Y}{1 + Y}$

Substitute **Y = e^(β₀ + β₁x)** in p(x) = Y / (1 + Y):

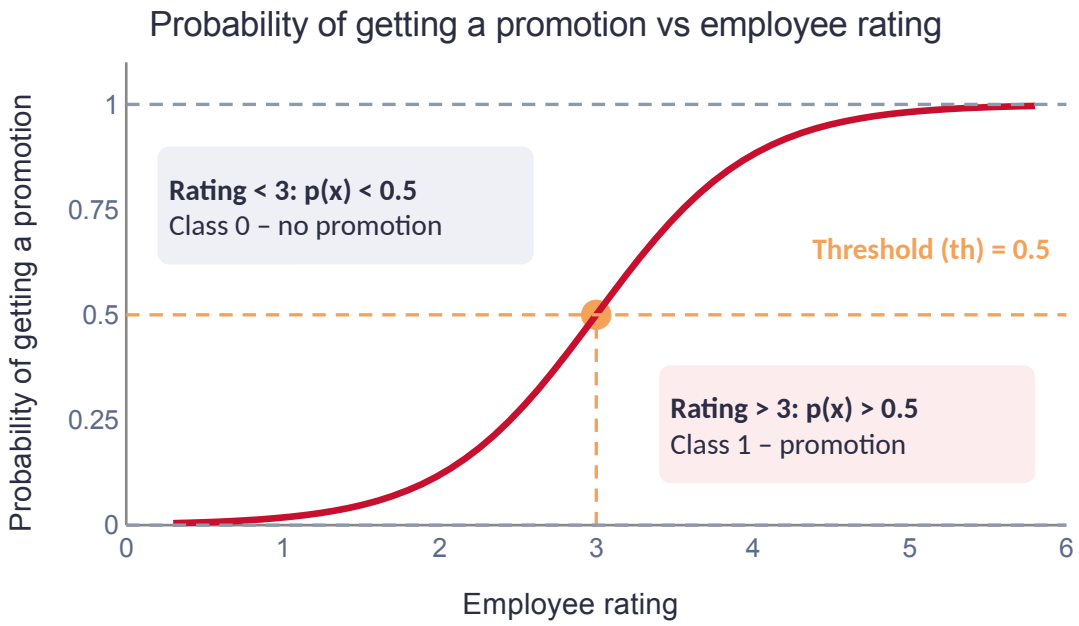
$$p(x) = \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}}$$

Dividing numerator and denominator by e^(β₀ + β₁x):

$$p(x) = \frac{1}{e^{-(\beta_0 + \beta_1 x)} + 1}$$

The Sigmoid Curve

The sigmoid curve obtained from the above equation is as follows:



$$p(x) = \frac{1}{1 + e^{-y}}$$

where $y = \beta_0 + \beta_1 x$

Output always between 0 and 1 so it can be read as a probability. The orange dashed line marks the threshold 0.5; it crosses the curve at rating 3, the decision boundary.

Understanding the Threshold

What it is

The threshold (th) is the cut-off probability that turns the sigmoid output p(x) into a class label. The usual default is 0.5.

Decision rule

If $p(x) \geq th \rightarrow$ Class 1 (promotion / selected / spam).
If $p(x) < th \rightarrow$ Class 0 (no promotion / not selected / not spam).

Decision boundary

With th = 0.5, p(x) = 0.5 exactly when $\beta_0 + \beta_1 x = 0$, i.e. $x = -\beta_0 / \beta_1$. In the promotion graph this is an employee rating of 3.

Choosing the value

A higher th (e.g. 0.7) is stricter: fewer positives and fewer false alarms. A lower th (e.g. 0.3) catches more positives but raises false alarms. Pick it based on the cost of each error.

Solved Example: Student Admission

- The student dataset has entrance marks based on the historic data of those who are selected or not selected.
- Based on the logistic regression, the values of the learnt parameters are $\beta_0 = 1$ and $\beta_1 = 8$.
- Assuming marks of x = 60, compute the resultant class.

$$p(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

$$\beta_0 + \beta_1 x = 1 + 8 \times 60 = 481$$

$$p(x) = \frac{1}{1 + e^{-(1 + 8 \times 60)}} = \frac{1}{1 + e^{-481}}$$

$$p(x) = \frac{1}{1 + e^{-481}} = 0.44$$

Substituting x = 60 into the sigmoid

Check the arithmetic: e^(−481) ≈ 0, so 1 / (1 + e^(−481)) ≈ 1.0, not 0.44. With the parameters as given, p ≈ 1 > 0.5 would mean "selected".

If we assume the threshold value as 0.5, then it is observed that 0.44 < 0.5; therefore, the candidate with marks 60 is not selected.

Linear vs Logistic Regression

Aspect	Linear Regression	Logistic Regression
Problem type	Regression – predicts a numerical response	Classification – predicts a categorical (binary) response
Output	Continuous value, e.g. salary hike	Probability p(x), e.g. chance of promotion
Output range	$-\infty$ to $+\infty$ (can exceed 0 and 1)	Always between 0 and 1
Curve	Straight line	S-shaped sigmoid curve
Equation	$y = \beta_0 + \beta_1 x$	$p(x) = 1 / (1 + e^{-(\beta_0 + \beta_1 x)})$; $\text{logit}(p) = \beta_0 + \beta_1 x$
Fitting method	Least squares (minimise squared error)	Maximum likelihood (minimise log loss)
Threshold	Not needed	Needed to assign the class (default 0.5)
Example	Salary hike from employee rating	Promotion yes/no; admitted or not

K-Nearest Neighbours (KNN)

A simple guide to the idea, the distance measures, choosing k, and the pros and cons



What Is KNN?

> *“If it walks like a duck and quacks like a duck, then it’s probably a duck.”*
> KNN labels something new by looking at the examples most similar to it.



Supervised

Learns from labelled examples (data whose answer is already known).



Classification & regression

Classification: the neighbours vote.
Regression: take the average of the neighbours’ values.



“Lazy” learner

No model is built in training. It just stores the data and does the work when a new point arrives.



k = how many neighbours

k is the number of nearest neighbours we look at. It is the main setting to choose.

How KNN Works: 5 Simple Steps



1

Store

Keep all the labelled training examples.

›

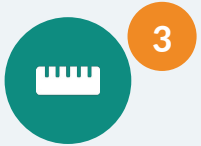


2

Choose k

Decide how many neighbours to look at (e.g. k = 3).

›

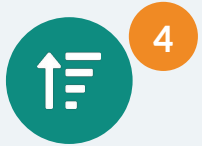


3

Measure

Find the distance from the new point to every training point.

›



4

Pick k nearest

Sort by distance and keep the k closest points.

›



5

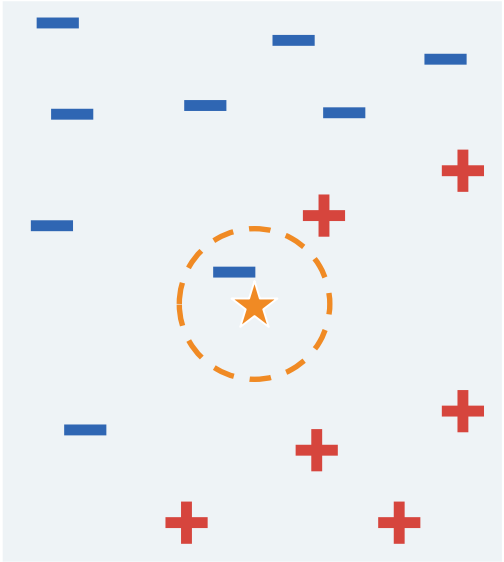
Vote

The most common label among them is the answer.

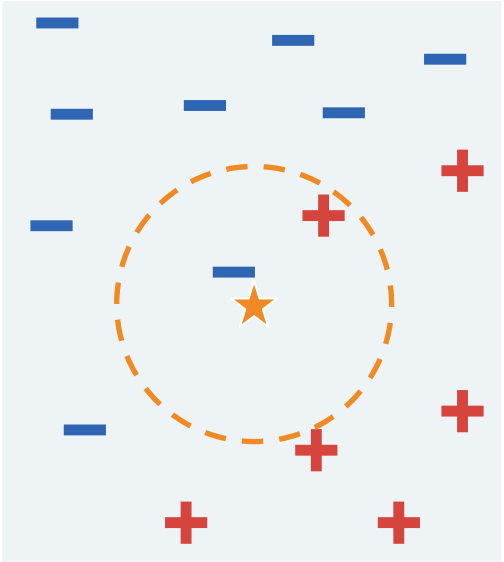
For regression (predicting a number, e.g. house price): instead of voting, take the average of the k neighbours' values.

Nearest Neighbours in a Picture

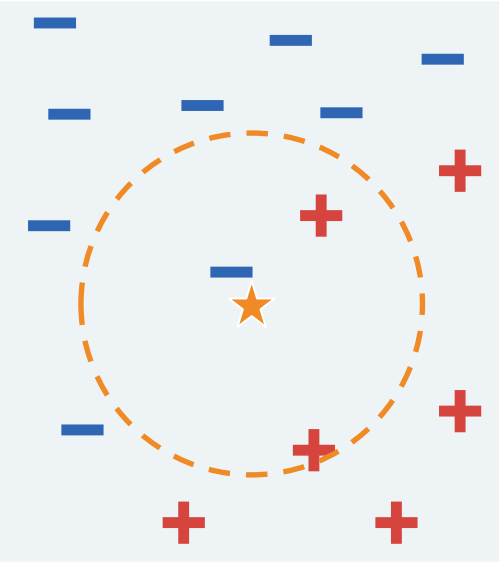
The k nearest neighbours of a new point ★ are the k training points closest to it. The dashed circle shows who gets a vote.



k = 1: Nearest point is "–"
→ **Class –**

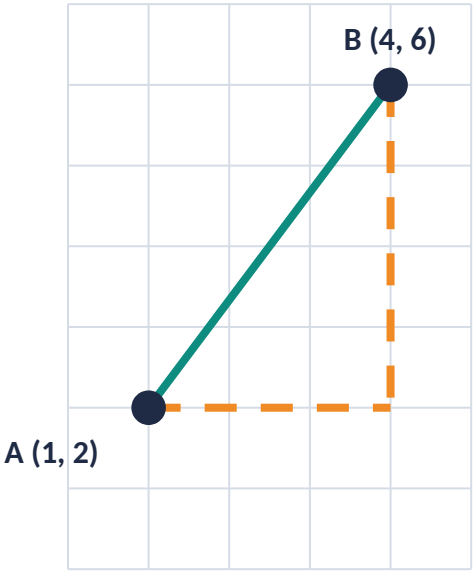


k = 2: One "–" and one "+"
→ **Tie! (why we use odd k)**



k = 3: Two "+" and one "–"
→ **Class +**

Distance Measures: How “Close” Is Close?



— Euclidean = 5
- - Manhattan = 3 + 4 = 7

Euclidean

Straight-line distance (“as the crow flies”). The most common choice.

$\sqrt{((x_1-y_1)^2 + (x_2-y_2)^2)}$

Manhattan

Walk along the grid, like city blocks. Add up the differences.

$|x_1-y_1| + |x_2-y_2|$

Minkowski

A general family: p = 1 gives Manhattan, p = 2 gives Euclidean.

one formula, adjustable p

Hamming

For categories (yes/no, colours): count how many features differ.

number of mismatches

Others exist too (Chebyshev, Canberra, Mahalanobis…), but Euclidean and Manhattan are enough to start.

Worked Example: T-Shirt Size

New customer: height 161 cm, weight 61 kg → size M or L?

Rank	Height	Weight	Size	Distance
1	163	61	M	2.00
2	160	59	M	2.24
3	160	64	L	3.16
4	163	64	L	3.61
5	165	61	L	4.00
6	158	58	M	4.24
7	157	64	M	5.00
8	165	65	L	5.66

Sorted by Euclidean distance to (161, 61). Green = 3 nearest, orange = next 2.

k = 3

Neighbours: M, M, L → 2 M vs 1 L

Prediction: M

k = 5

Neighbours: M, M, L, L, L → 2 M vs 3 L

Prediction: L

Lesson: same data, different k → different answer. Choosing k matters!

Choosing the Value of k

k too small (e.g. k = 1)

Listens to just one or two points, so one noisy or wrong point can change the answer. Boundary is jagged (overfitting).

k just right

Enough neighbours to ignore noise, but still local enough to follow the real pattern.

k too large (e.g. k = N)

The neighbourhood pulls in points from other classes; the answer drifts to the biggest class (underfitting).

Practical tips

1. **Use an odd k** for two classes, so votes can't tie.
2. **Start with k ≈ √N,** where N = number of training points (e.g. 100 points → k ≈ 10, use 9 or 11).
3. **Try a few values** and keep the one that does best on test (validation) data.

Advantages and Disadvantages

Advantages

- Very simple to understand and implement
- No training phase: just store the data
- Works for classification and regression
- Handles many classes naturally
- Can follow curvy, non-linear boundaries
- New data can be added any time

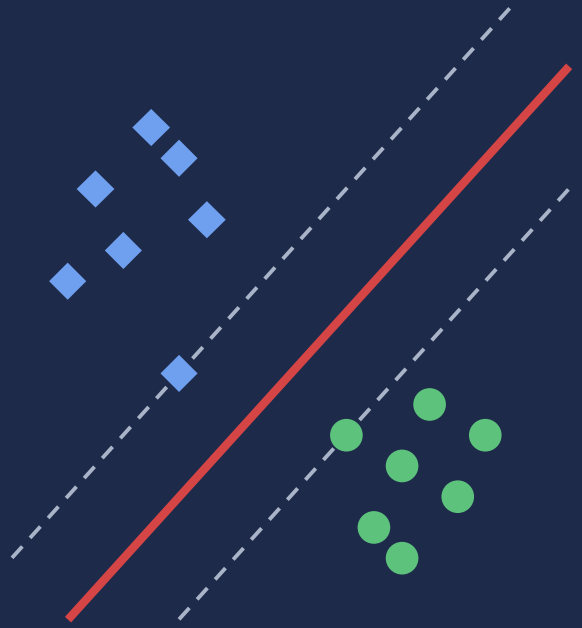
Disadvantages

- Slow to predict on big datasets (compares with every point)
- Needs memory to store all the data
- Must pick a good k and distance measure
- Sensitive to noise and irrelevant features
- Needs feature scaling (see tip below)
- Struggles with many features (curse of dimensionality)

Tip – scale your features: if salary (₹10,000–₹1,00,000) and age (20–60) are used together, salary dominates every distance and age is ignored. Bring all features to a similar range first.

Support Vector Machine (SVM)

Hyperplane, margin, support vectors, linear and non-linear SVM, and the kernel trick

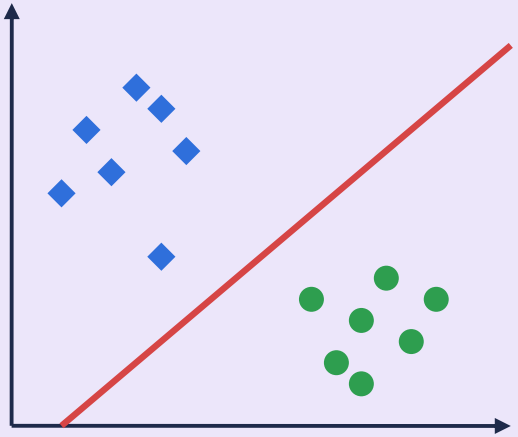


What Is SVM?

- Support Vector Machine (SVM) is one of the most popular Supervised Learning algorithms, used for Classification as well as Regression problems.
- However, it is primarily used for Classification problems in Machine Learning.
- The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes, so that we can easily put a new data point in the correct category in the future.
- This best decision boundary is called a hyperplane.

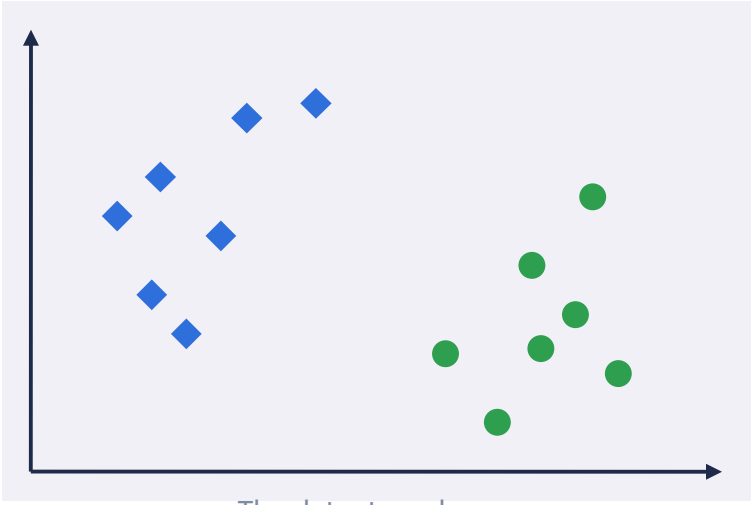
In short

SVM finds the hyperplane that best separates the classes with the maximum margin.

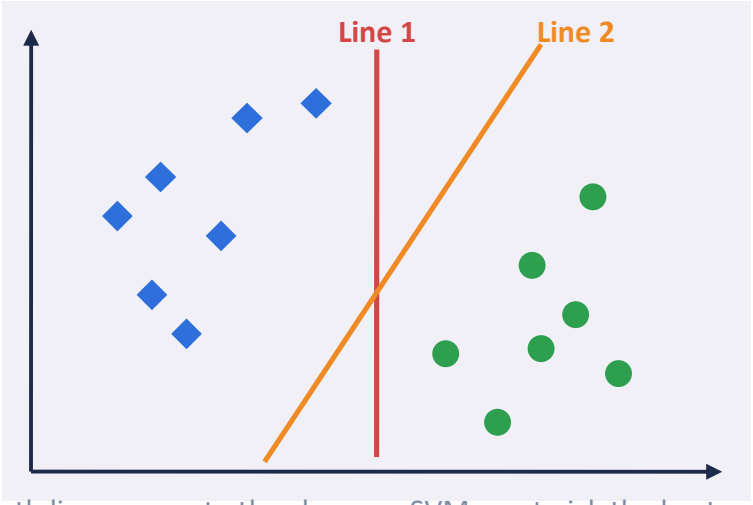


Many Possible Boundaries – Which Is Best?

- There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find the best decision boundary that helps to classify the data points.
- This best boundary is known as the hyperplane of SVM.



The data: two classes



Both lines separate the classes – SVM must pick the best one

Key Concepts

Hyperplane

The decision boundary that separates the classes.

Support Vectors

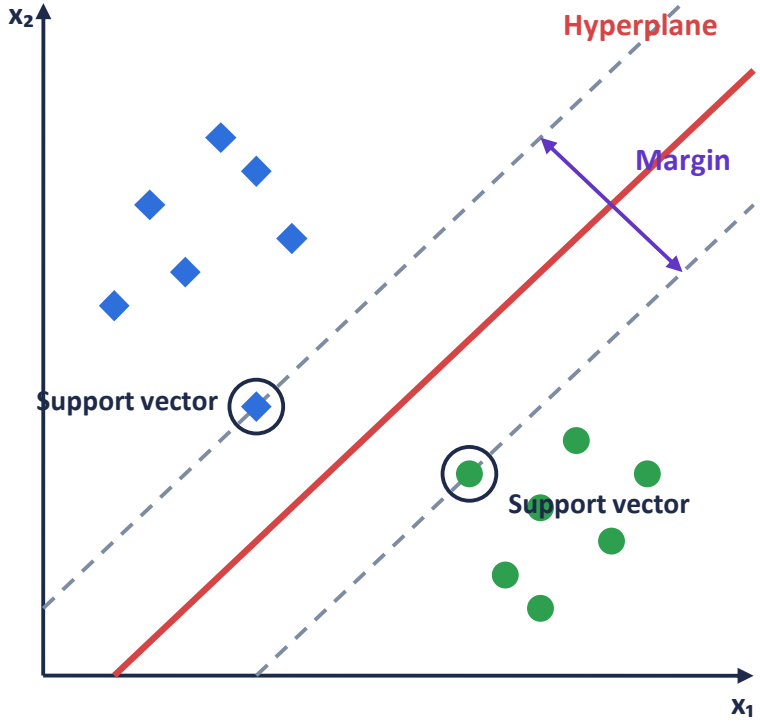
The data points nearest to the boundary.

Margin

The distance from the boundary to the support vectors.

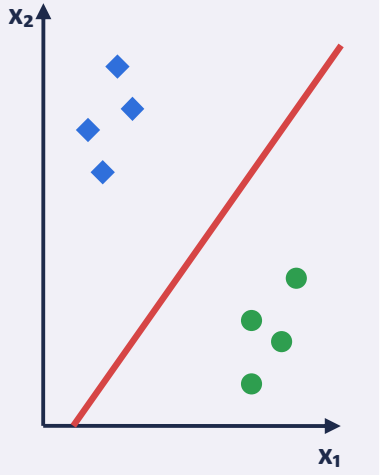
Goal

Maximise the margin.

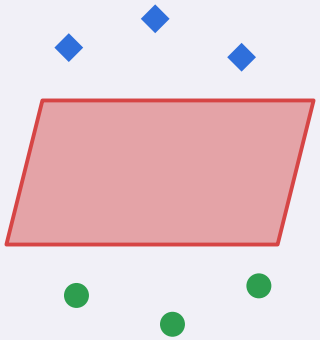


Hyperplane

- The dimensions of the hyperplane depend on the features present in the dataset: if there are 2 features, the hyperplane will be a straight line.
- If there are 3 features, the hyperplane will be a 2-dimensional plane.
- We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.



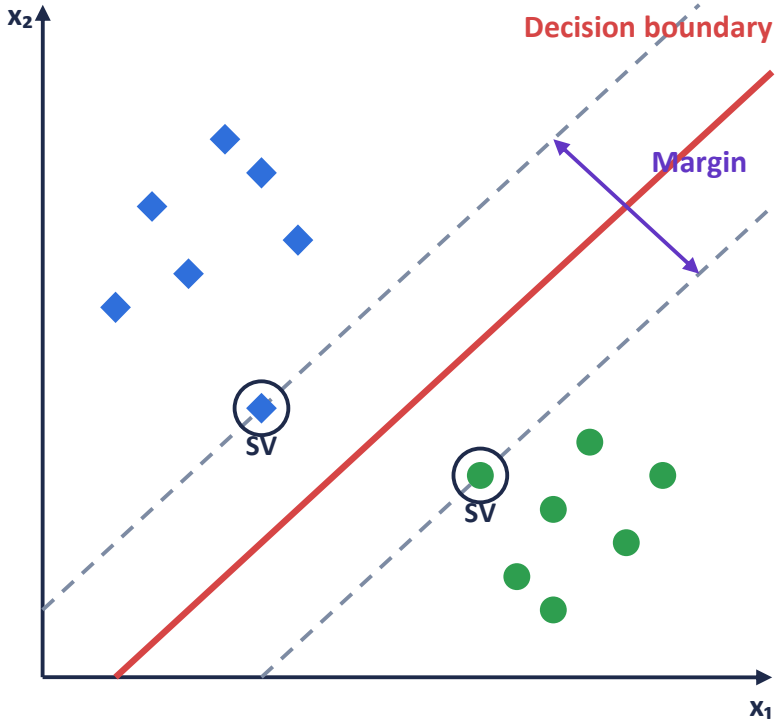
2 features
hyperplane = a line



3 features
hyperplane = a 2-D plane

In n dimensions, the hyperplane has n − 1 dimensions.

Support Vectors and the Optimal Hyperplane



- The data points or vectors closest to the hyperplane, which affect its position, are termed Support Vectors.
- Since these vectors support the hyperplane, they are called Support Vectors.

Optimal hyperplane

- The hyperplane that maximises the margin between the two classes.
- Dashed lines = margin boundaries.
- Points on the dashed lines = support vectors.
- The distance between support vectors and hyperplane should be as large as possible.

How SVM Works: Step by Step

1 **Plot the data**
Each example is a point in n-dimensional space (one axis per feature).

2 **Draw candidate boundaries**
Many lines/planes may separate the two classes.

3 **Find the support vectors**
For each boundary, find the points of each class closest to it.

4 **Measure the margin**
The gap between the boundary and the support vectors on both sides.

5 **Pick the maximum margin**
Choose the hyperplane with the largest margin – the optimal hyperplane.

6 Outliers → Soft margin (allow some errors)
Non-linear data → Kernel trick (transform to higher dimension)

7 **Classify new points**
A new point gets the class of the side of the hyperplane it falls on.

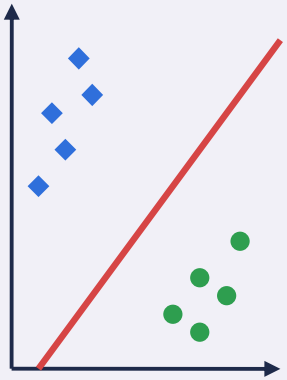
Rule: $w \cdot x + b \geq 0 \rightarrow$ class 1; $w \cdot x + b < 0 \rightarrow$ class 2

Types of SVM

SVM can be of two types:

1. Linear SVM

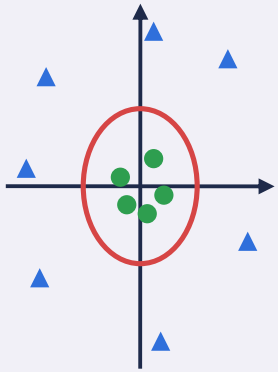
Used for linearly separable data: if a dataset can be classified into two classes using a single straight line, the data is termed linearly separable, and the classifier used is called a Linear SVM classifier.



One straight line works

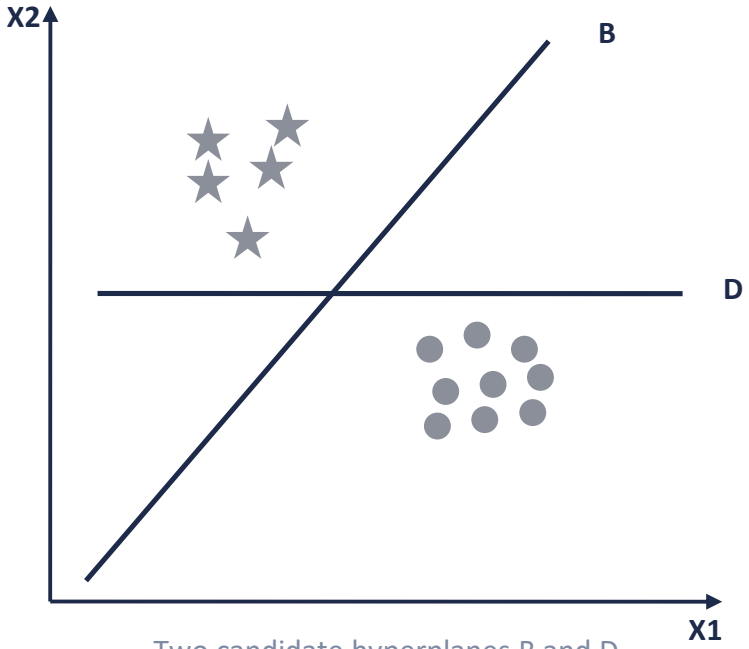
2. Non-linear SVM

Used for non-linearly separated data: if a dataset cannot be classified using a straight line, the data is termed non-linear data, and the classifier used is called a Non-linear SVM classifier.

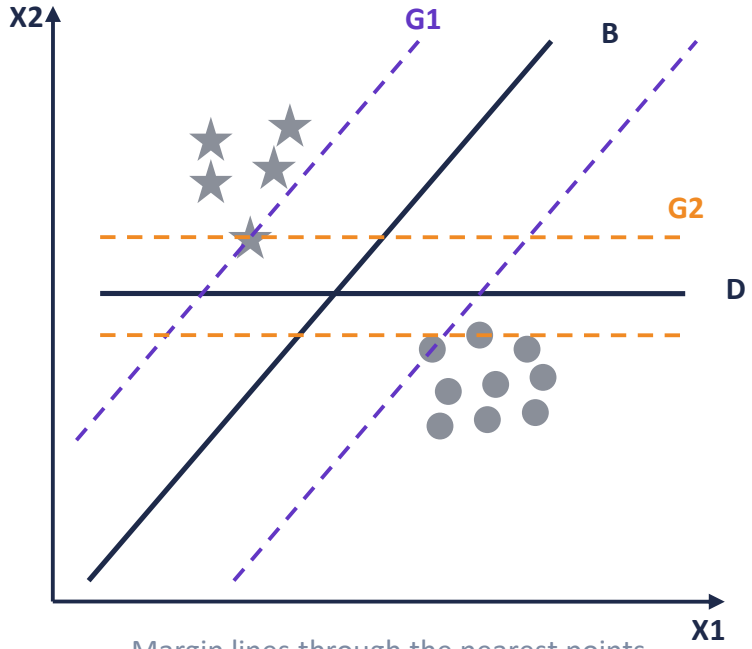


No straight line works

Linear SVM: Choosing the Best Hyperplane



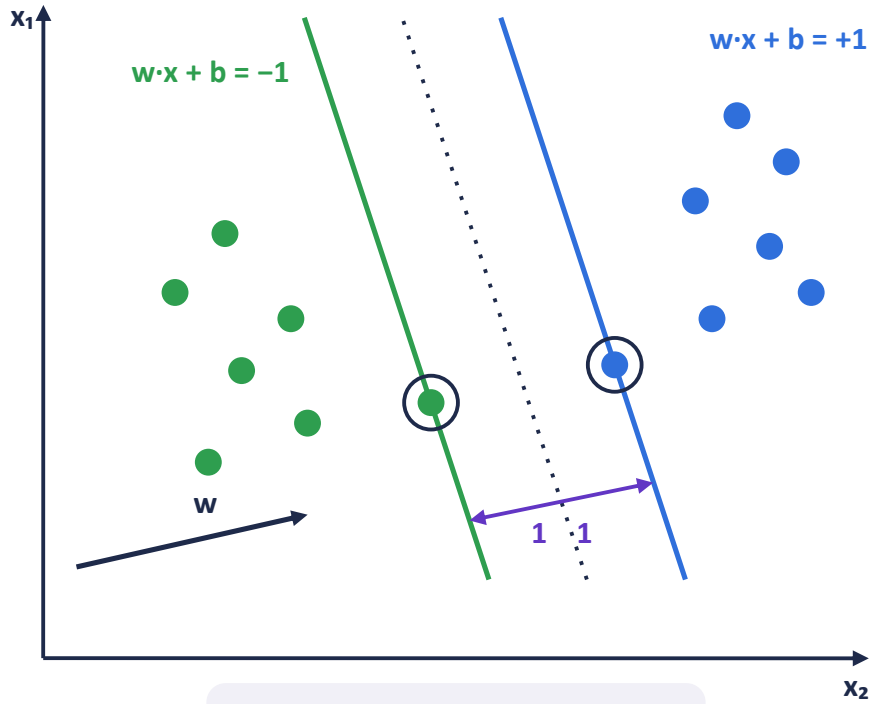
Two candidate hyperplanes B and D



Margin lines through the nearest points

Margin of B > margin of D → **B is the better hyperplane.**

The Margin in Equations



w·x + b = 0 (hyperplane)

SVM only allows hyperplanes that satisfy $y_i(w \cdot x_i + b) \geq 1$ for all points.This just says positive points must have $w \cdot x + b \geq +1$ and negative points must have $w \cdot x + b \leq -1$.

SVM optimisation problem

$$\max \frac{2}{\|\vec{w}\|}$$

$$\text{s.t. } (\vec{w} \cdot \vec{x} + b) \geq 1, \quad \forall x \text{ of class 1}$$

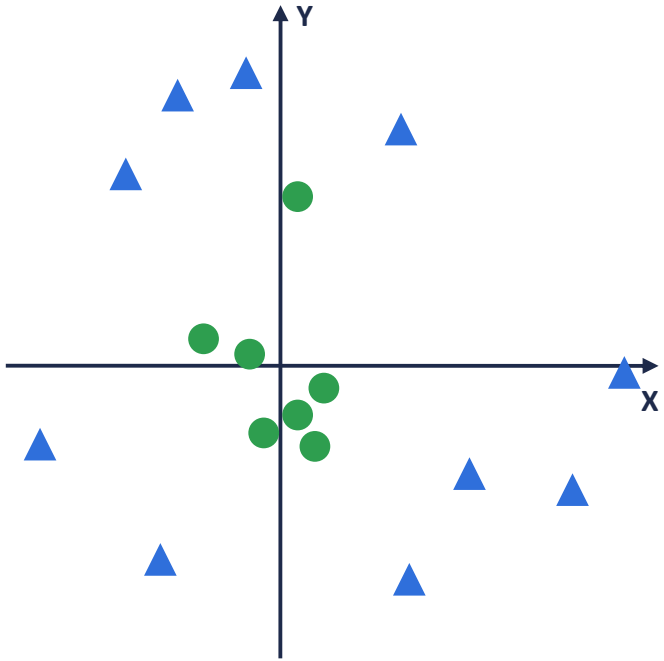
$$(\vec{w} \cdot \vec{x} + b) \leq -1, \quad \forall x \text{ of class 2}$$

- Hyperplane: $w \cdot x + b = 0$; margin lines: $w \cdot x + b = +1$ and -1.
- Each side of the margin is $1/\|w\|$, so the total margin = $2/\|w\|$.
- Maximising $2/\|w\|$ (minimising $\|w\|$) gives the widest margin while every point stays on its correct side.

Non-Linear SVM

- If data is linearly arranged, we can separate it using a straight line, but for non-linear data we cannot draw a single straight line.
- So to separate these data points, we need to add one more dimension.
- For linear data we used two dimensions x and y, so for non-linear data we will add a third dimension z.
- It can be calculated as:

$$z = x^2 + y^2$$



Green points sit in the middle, surrounded by blue – no straight line can separate them

Kernel Trick?

Definition

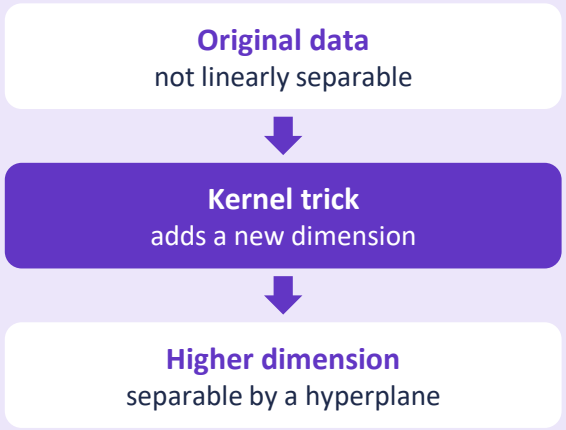
The kernel trick is a shortcut that lets SVM separate data that no straight line can split. It lifts the data into a higher dimension, where a flat boundary can separate the classes.

Purpose

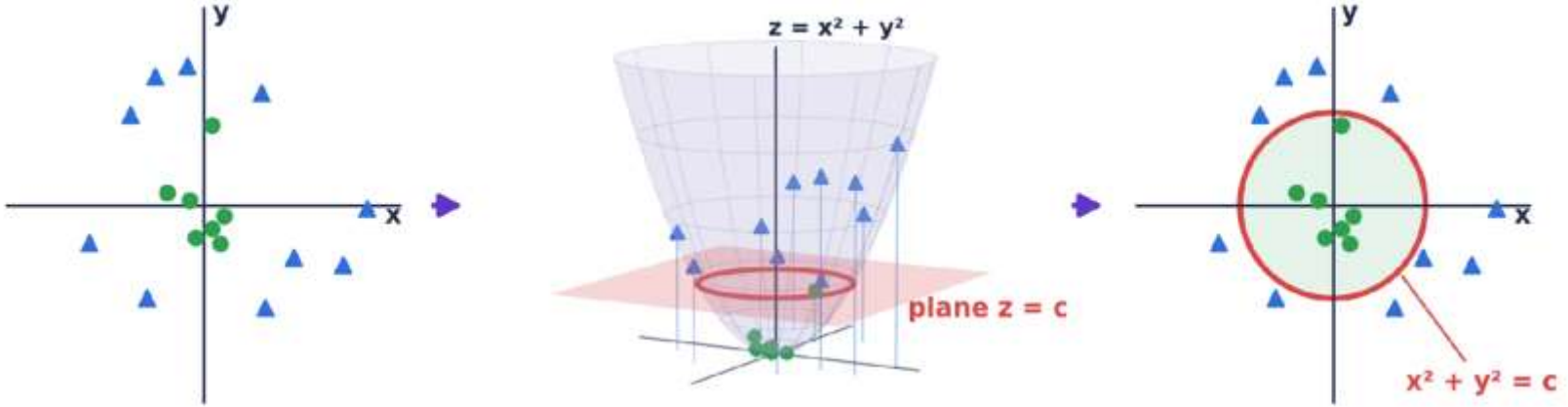
- **Handle non-linear data:** it turns a curved separation problem into a straight-line one.
- **Save time and memory:** SVM does not need to build and store the extra features.
- **Keep SVM simple:** SVM still finds the maximum-margin hyperplane, just in the new space.

In short

Lift the data so a straight boundary can separate it, without the heavy calculation.



The Kernel Trick: Adding $z = x^2 + y^2$



1 Original (x, y)

No straight line separates the classes

2 Map $\phi(x, y) = (x, y, x^2 + y^2)$

Points rise onto a bowl; a flat plane separates

3 Back in (x, y)

The plane z = c becomes a circle

What happens

Green points lie near the origin, so $z = x^2 + y^2$ is small; blue points lie far out, so z is large. A linear SVM in (x, y, z) finds the plane z = c, which is the circle $x^2 + y^2 = c$ in the original data.

Why it is a "trick"

SVM only needs dot products between points. A kernel returns $\phi(a)\cdot\phi(b)$ directly, here $K(a, b) = a\cdot b + \|a\|^2\|b\|^2$, so the new features are never built. Common kernels: polynomial, RBF.

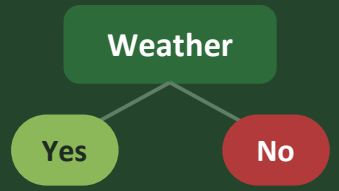
Advantages and Limitations

Advantages

- Works well in high-dimensional spaces (many features), e.g. text and images
- Effective even when features outnumber the samples
- Memory efficient: only the support vectors define the hyperplane
- Kernel trick handles non-linear boundaries
- Maximum margin gives good generalisation; soft margin tolerates outliers

Limitations

- Slow to train on very large datasets
- Struggles when classes overlap heavily or data is noisy
- Needs careful choice of kernel and its parameters (C, γ)
- Needs feature scaling (e.g. salary vs age)
- No direct probability output, and harder to interpret than a decision tree

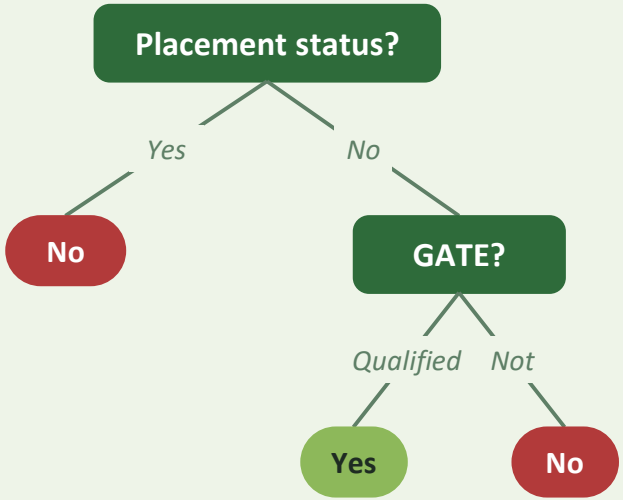


Decision Tree Classification

What is a decision tree?

- A supervised learning model that predicts an outcome by asking a sequence of questions about the features.
- Used for both classification (Yes/No, class labels) and regression (numeric values).
- Each answer moves you down one branch until you reach a final decision.
- Easy to read and explain — the model is literally a flowchart of if/else rules.

Example: Should I continue study in a college?



Anatomy of a decision tree

1

Root node

The first split — the attribute with the highest information gain.

2

Decision node

An internal test on an attribute (drawn as a box).

3

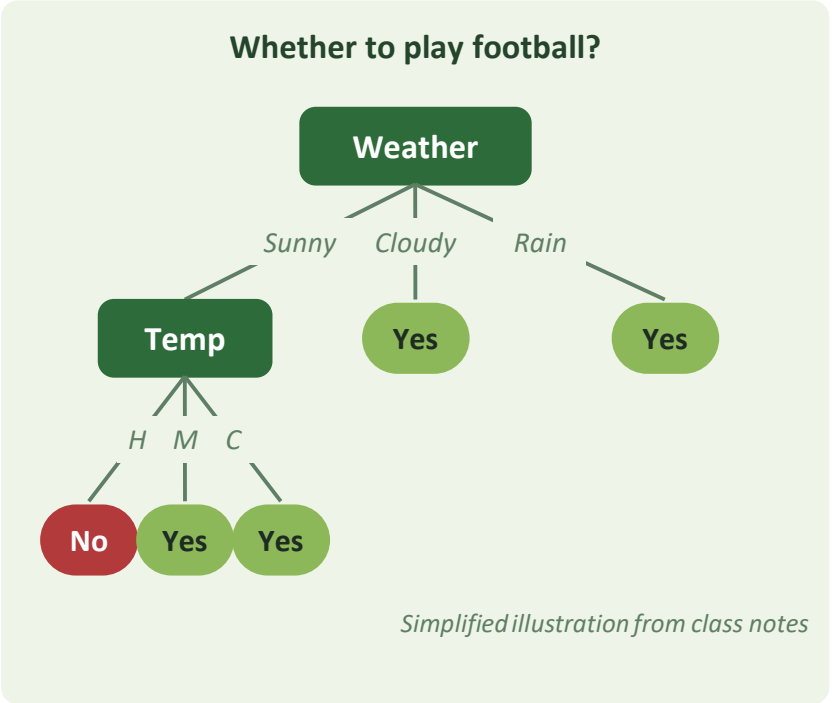
Branch

One possible value of the attribute being tested.

4

Leaf node

The final outcome; ideally a pure class (all Yes or all No).



Simplified illustration from class notes

Measuring purity: Entropy and Info Gain

Entropy

$$E(S) = -\,p_{+}\log_2 p_{+} - p_{-}\log_2 p_{-}$$

How mixed a set is. 0 = pure (all one class); 1 = a 50/50 split between two classes.

Information Gain

$$IG(S, A) = E(S) - \Sigma\,(|S_v| \,/\, |S|) \cdot E(S_v)$$

The drop in entropy after splitting on attribute A. Higher IG = purer children = better split.

1. Compute entropy of the dataset
2. Compute IG for every attribute
3. Split on the highest-IG attribute
4. Repeat on each branch until leaves are pure

The ID3 algorithm

The dataset: Should we play football?

Day	Weather	Temp	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Cloudy	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Cloudy	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Cloudy	Mild	High	Strong	Yes
13	Cloudy	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

14 days · 9 Yes · 5 No

Steps to build the tree

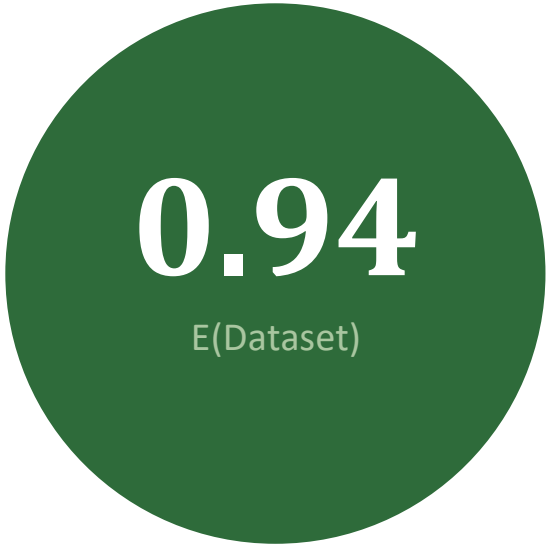
1. Find the entropy of the dataset
2. Find the IG of every attribute
3. Highest IG becomes the root node
4. Make a new dataset for each branch
5. Repeat IG on each new dataset
6. Stop when every leaf is pure

Step 1: Entropy of the whole dataset

S = {+9, −5} (9 Yes, 5 No out of 14)

$$E(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14)$$
$$= 0.410 + 0.530$$
$$= \mathbf{0.940}$$

The dataset is quite mixed (close to 1), so there is plenty of uncertainty for a good split to remove.



Step 2: Information Gain of Weather

Sunny

5 days · {+2, −3}

$-(2/5)\log_2(2/5) - (3/5)\log_2(3/5)$

E = 0.971

Cloudy

4 days · {+4, −0}

$-(4/4)\log_2(4/4)$

E = 0 (pure)

Rain

5 days · {+3, −2}

$-(3/5)\log_2(3/5) - (2/5)\log_2(2/5)$

E = 0.971

$$IG(S, Weather) = E(S) - (5/14)\cdot E(Sunny) - (4/14)\cdot E(Cloudy) - (5/14)\cdot E(Rain)$$

$$= 0.940 - (5/14)(0.971) - (4/14)(0) - (5/14)(0.971) = \mathbf{0.246}$$

Step 3: Information Gain of Temperature

Hot

4 days · {+2, −2}

$-(2/4)\log_2(2/4) - (2/4)\log_2(2/4)$

E = 1.000

Mild

6 days · {+4, −2}

$-(4/6)\log_2(4/6) - (2/6)\log_2(2/6)$

E = 0.918

Cool

4 days · {+3, −1}

$-(3/4)\log_2(3/4) - (1/4)\log_2(1/4)$

E = 0.811

$$IG(S, Temp) = E(S) - (4/14)\cdot E(Hot) - (6/14)\cdot E(Mild) - (4/14)\cdot E(Cool)$$

$$= 0.940 - (4/14)(1.000) - (6/14)(0.918) - (4/14)(0.811) = \mathbf{0.029}$$

Step 4: Information Gain of Humidity

High

7 days · {+3, −4}

$-(3/7)\log_2(3/7) - (4/7)\log_2(4/7)$

E = 0.985

Normal

7 days · {+6, −1}

$-(6/7)\log_2(6/7) - (1/7)\log_2(1/7)$

E = 0.592

$$IG(S, Humidity) = E(S) - (7/14)\cdot E(High) - (7/14)\cdot E(Normal)$$

$$= 0.940 - (7/14)(0.985) - (7/14)(0.592) = \mathbf{0.152}$$

Step 5: Information Gain of Wind

Weak

8 days · {+6, −2}

$-(6/8)\log_2(6/8) - (2/8)\log_2(2/8)$

E = 0.811

Strong

6 days · {+3, −3}

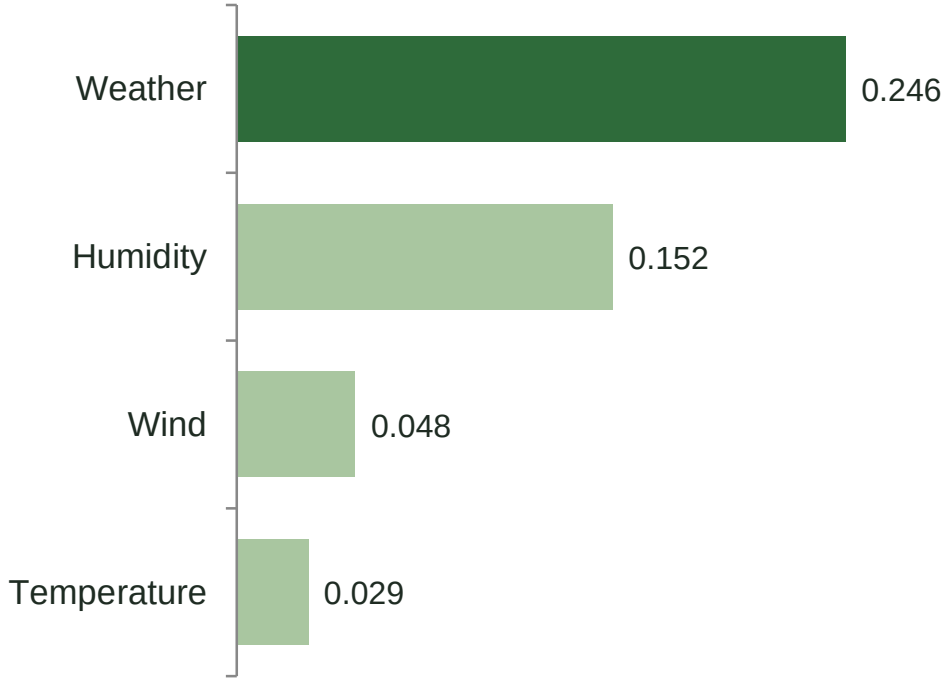
$-(3/6)\log_2(3/6) - (3/6)\log_2(3/6)$

E = 1.000

$IG(S, Wind) = E(S) - (8/14)\cdot E(Weak) - (6/14)\cdot E(Strong)$

$= 0.940 - (8/14)(0.811) - (6/14)(1.000) =$ **0.048**

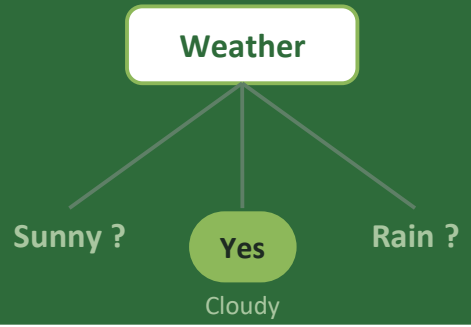
Step 6: Compare gains and select the root



Max IG = 0.246

Weather is the root node

It removes the most uncertainty, and its Cloudy branch is already pure (all Yes).



Next node: the new dataset for Sunny

Day	Weather	Temp	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes

Only the 5 Sunny days are kept. Weather is already used, so the candidates are Temperature, Humidity and Wind.

5 Sunny days

2 Yes · 3 No

$$E(\text{Sunny}) = -(2/5)\log_2(2/5) - (3/5)\log_2(3/5)$$

= 0.971

Sunny: Information Gain of Temperature

Hot

Days 1, 2 · {+0, −2}

$-(2/2)\log_2(2/2)$

E = 0 (pure)

Mild

Days 8, 11 · {+1, −1}

$-(1/2)\log_2(1/2) - (1/2)\log_2(1/2)$

E = 1.000

Cool

Day 9 · {+1, −0}

$-(1/1)\log_2(1/1)$

E = 0 (pure)

$$IG(Sunny, Temp) = E(Sunny) - (2/5)\cdot E(Hot) - (2/5)\cdot E(Mild) - (1/5)\cdot E(Cool)$$

$$= 0.971 - (2/5)(0) - (2/5)(1.000) - (1/5)(0) = \mathbf{0.571}$$

Sunny: Information Gain of Humidity

High

Days 1, 2, 8 · {+0, −3}

$-(3/3)\log_2(3/3)$

E = 0 (pure)

Normal

Days 9, 11 · {+2, −0}

$-(2/2)\log_2(2/2)$

E = 0 (pure)

$$IG(Sunny, Humidity) = E(Sunny) - (3/5)\cdot E(High) - (2/5)\cdot E(Normal)$$

$$= 0.971 - (3/5)(0) - (2/5)(0) = \mathbf{0.971}$$

Sunny: Information Gain of Wind

Weak

Days 1, 8, 9 · {+1, −2}

$-(1/3)\log_2(1/3) - (2/3)\log_2(2/3)$

E = 0.918

Strong

Days 2, 11 · {+1, −1}

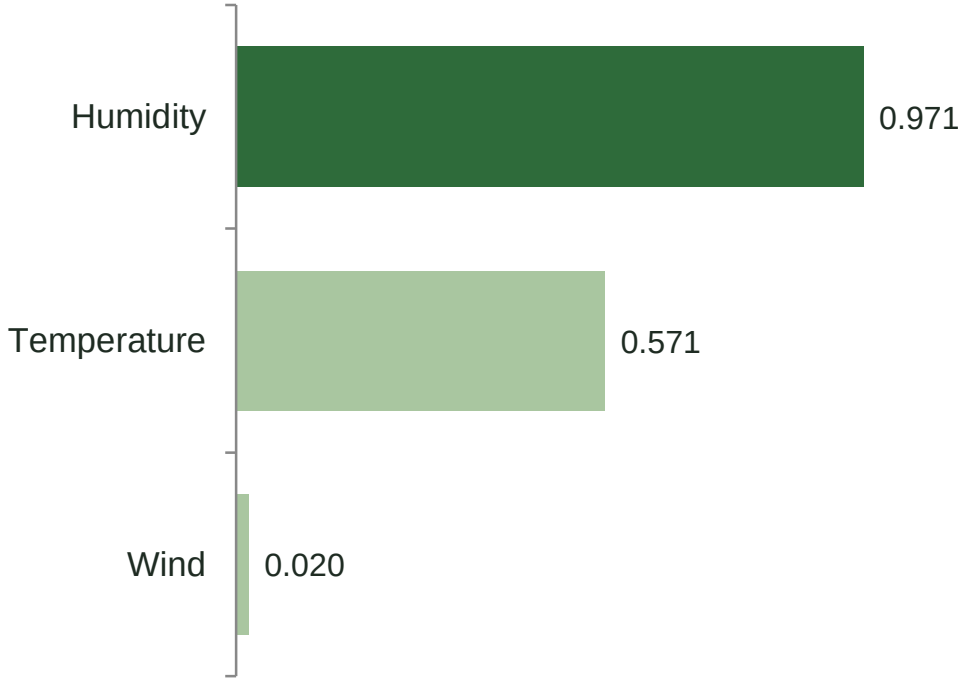
$-(1/2)\log_2(1/2) - (1/2)\log_2(1/2)$

E = 1.000

$$IG(Sunny, Wind) = E(Sunny) - (3/5)\cdot E(Weak) - (2/5)\cdot E(Strong)$$

$$= 0.971 - (3/5)(0.918) - (2/5)(1.000) = \mathbf{0.020}$$

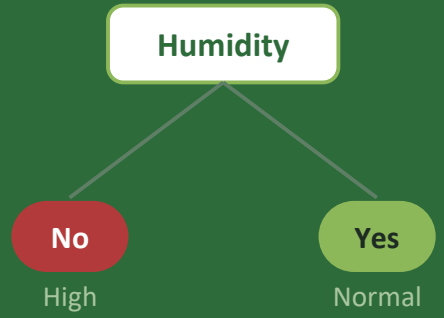
Sunny: compare gains and split on Humidity



Max IG = 0.971

Split Sunny on Humidity

Both branches are pure, so they become leaf nodes straight away.



Next node: the new dataset for Rain

Day	Weather	Temp	Humidity	Wind	Play?
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
10	Rain	Mild	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Only the 5 Rain days are kept. Weather is already used, so the candidates are Temperature, Humidity and Wind. (Cloudy needs no split: it is already pure, all Yes.)

5 Rain days

3 Yes · 2 No

$$E(\text{Rain}) = -(3/5)\log_2(3/5) - (2/5)\log_2(2/5)$$

= 0.971

Rain: Information Gain of Temperature

Hot

No Rain days · {+0, −0}

Empty subset, weight 0/5

E = 0 (no data)

Mild

Days 4, 10, 14 · {+2, −1}

$-(2/3)\log_2(2/3) - (1/3)\log_2(1/3)$

E = 0.918

Cool

Days 5, 6 · {+1, −1}

$-(1/2)\log_2(1/2) - (1/2)\log_2(1/2)$

E = 1.000

$$IG(Rain, Temp) = E(Rain) - (0/5)\cdot E(Hot) - (3/5)\cdot E(Mild) - (2/5)\cdot E(Cool)$$

$$= 0.971 - 0 - (3/5)(0.918) - (2/5)(1.000) = \mathbf{0.020}$$

Rain: Information Gain of Humidity

High

Days 4, 14 · {+1, −1}

$-(1/2)\log_2(1/2) - (1/2)\log_2(1/2)$

E = 1.000

Normal

Days 5, 6, 10 · {+2, −1}

$-(2/3)\log_2(2/3) - (1/3)\log_2(1/3)$

E = 0.918

$$IG(Rain, Humidity) = E(Rain) - (2/5)\cdot E(High) - (3/5)\cdot E(Normal)$$

$$= 0.971 - (2/5)(1.000) - (3/5)(0.918) = \mathbf{0.020}$$

Rain: Information Gain of Wind

Weak

Days 4, 5, 10 · {+3, −0}

$-(3/3)\log_2(3/3)$

E = 0 (pure)

Strong

Days 6, 14 · {+0, −2}

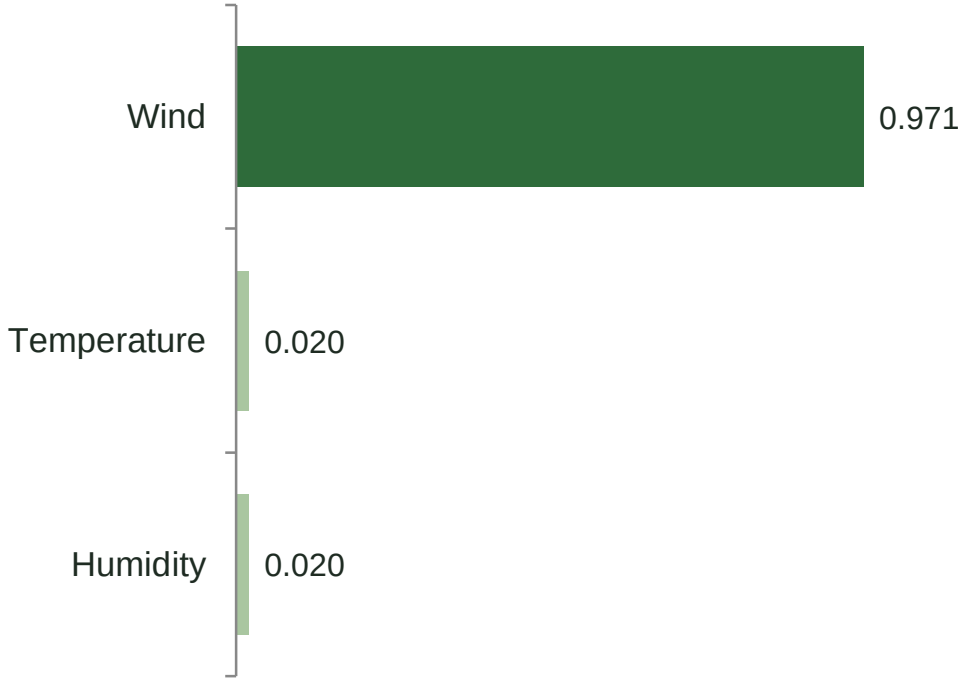
$-(2/2)\log_2(2/2)$

E = 0 (pure)

$$IG(Rain, Wind) = E(Rain) - (3/5)\cdot E(Weak) - (2/5)\cdot E(Strong)$$

$$= 0.971 - (3/5)(0) - (2/5)(0) = \mathbf{0.971}$$

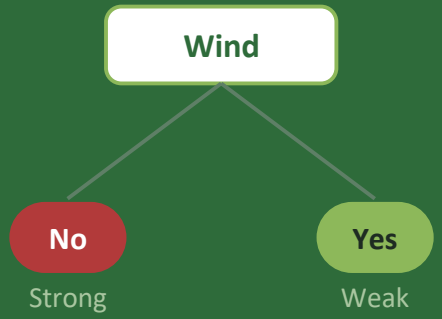
Rain: compare gains and split on Wind



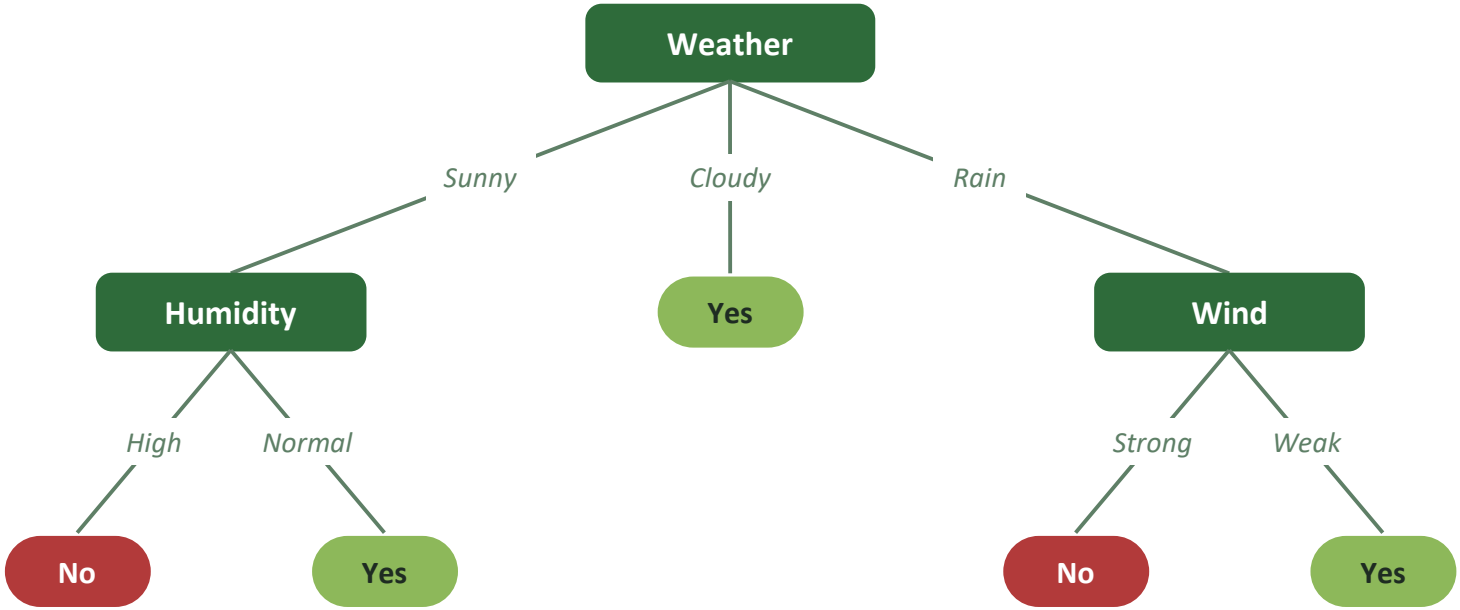
Max IG = 0.971

Split Rain on Wind

Both branches are pure, so they become leaf nodes straight away.



The final decision tree



Every leaf is pure, so the tree classifies all 14 training days correctly. Temperature was never needed.

Exercise: decision tree for A = (X AND Y) OR Z

X	Y	Z	A
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

5 True, 3 False

$E(S) = -(5/8)\log_2(5/8) - (3/8)\log_2(3/8) = 0.954$

1. Root: IG on all 8 rows

Z=1 {4T, 0F}: E=0 · Z=0 {1T, 3F}: E=0.811

IG(Z) = 0.954 – 0 – (4/8)(0.811) = 0.549

X=1 {3T,1F}: 0.811 · X=0 {2T,2F}: 1.0

IG(X) = IG(Y) = 0.954 – 0.406 – 0.5 = 0.049

Highest IG → Z is the root

2. Branch Z = 0 (4 rows: 1T, 3F)

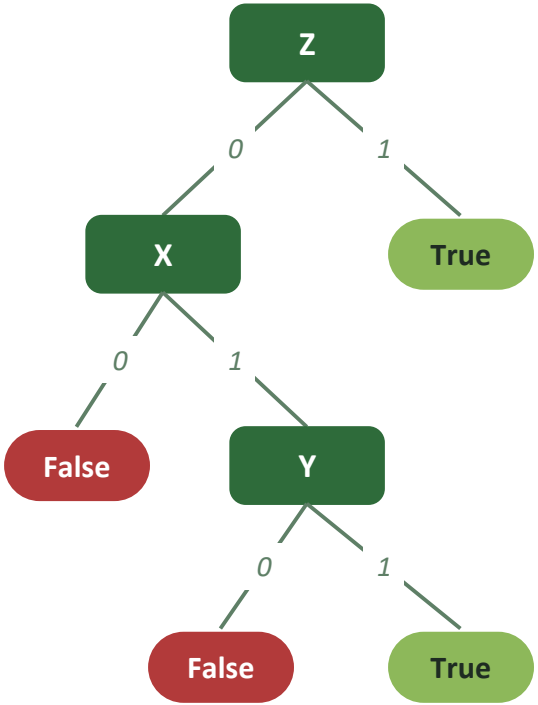
$E(Z=0) = -\frac{1}{4}\log_2\frac{1}{4} - \frac{3}{4}\log_2\frac{3}{4} = 0.811$

X=1 {1T,1F}: 1.0 · X=0 {0T,2F}: 0

IG(X) = IG(Y) = 0.811 – (2/4)(1.0) = 0.311

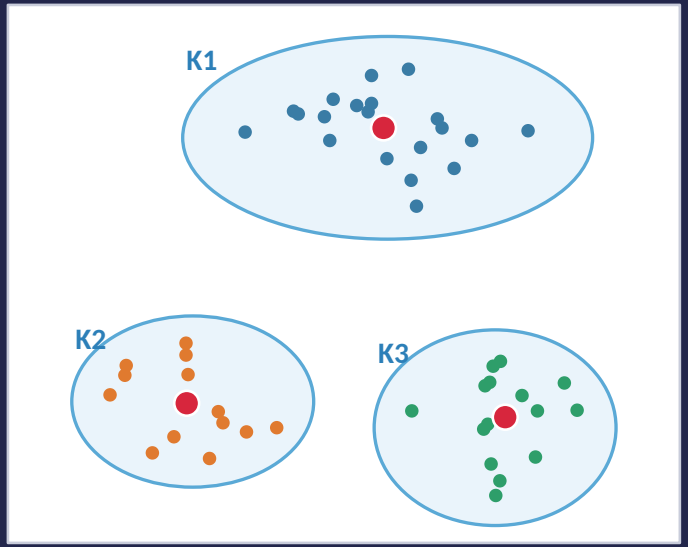
Tie → choose X; X=0 is pure (False)

X=1 → split on Y: Y=1 True, Y=0 False



K-Means Clustering

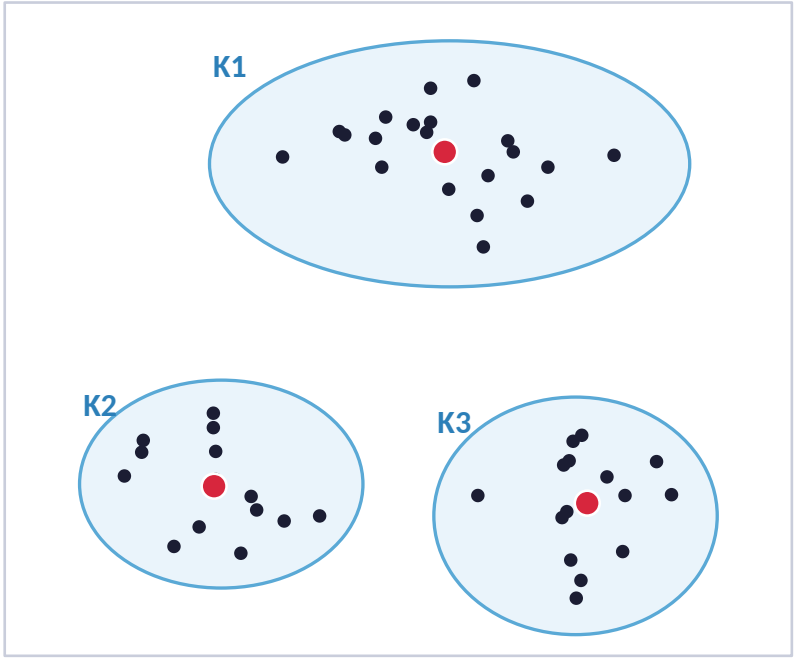
An unsupervised algorithm that groups similar data points into k clusters



What Is K-Means Clustering?

K-means clustering is a **distance-based unsupervised clustering algorithm** where data points that are close to each other are grouped into a given number of clusters/groups.

Algorithm categorizes the items into **k groups or clusters of similarity**.



k = 3 clusters (K1, K2, K3); red dot = centroid (cluster centre)

Measuring Similarity

To calculate the similarity, we can use the Euclidean distance, Manhattan distance, Hamming distance or Cosine distance as the measurement. Close points = similar points.

Euclidean

Straight-line distance between two points. The usual choice for K-means.

$\sqrt{((x_1-y_1)^2 + (x_2-y_2)^2)}$

Manhattan

Distance along the grid (city blocks): add up the differences.

$|x_1-y_1| + |x_2-y_2|$

Hamming

For categorical/binary features: count how many features differ.

number of mismatches

Cosine

Compares the direction (angle) of two vectors, not their size. Common for text.

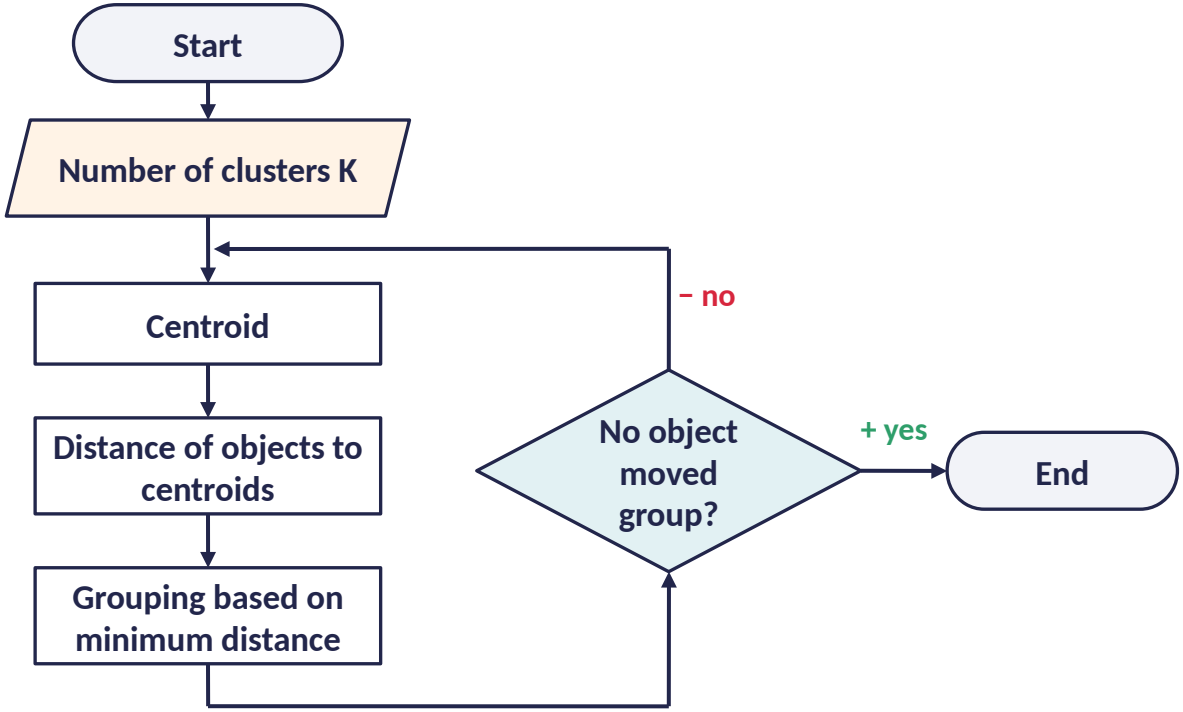
1 − cos(angle)

K-Means Algorithm: Pseudocode

Input: Algorithm K-Means (K = number of clusters, D = list of data points)

1. Choose K random data points as the initial centroids (cluster centres).
2. **Repeat until the cluster centres stabilize:**
   - a. Allocate each point in D to the nearest of the K centroids. *(Assignment step)*
   - b. Compute the centroid of each cluster using all the points in the cluster. *(Update step – the centroid is the mean of its points)*

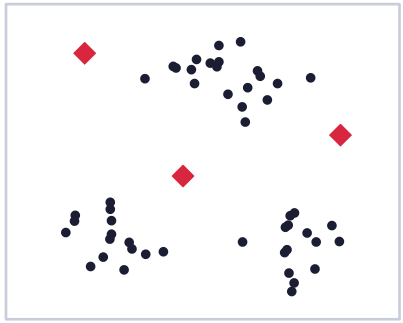
K-Means Algorithm: Flowchart



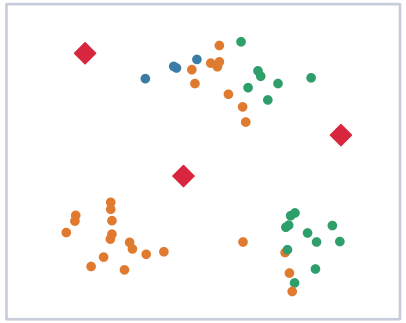
Reading the flowchart

- Choose K and the starting centroids.
- Measure every point's distance to each centroid.
- Put each point in the group of its nearest centroid.
- Did any point change group? Yes → recompute the centroids and loop again.
- No point moved → the clusters are stable: stop.

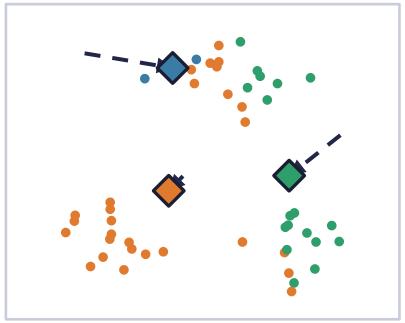
K-Means in Action (k = 3)



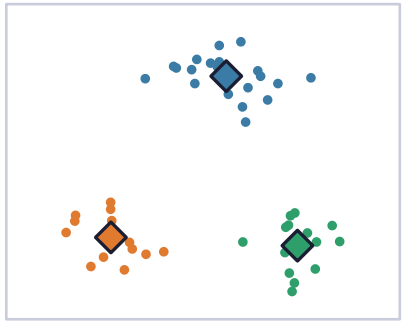
1. Start
Pick 3 random centroids (red).



2. Assign
Each point joins its nearest centroid.



3. Update
Move each centroid to the mean of its points.



4. Repeat → done
Re-assign and update until no point changes group (4 rounds here).

What to notice: the centroids start in poor places, but each round of "assign → update" pulls them towards the real groups. When the centroids stop moving, the algorithm has converged and the final clusters match K1, K2 and K3.

Solved Example: The Problem

Let us run K-means by hand on a small data set of 8 points.

Task: group 8 points into **k = 3** clusters

A1 (2, 10)	**A2** (2, 5)	**A3** (8, 4)	**B1** (5, 8)
B2 (7, 5)	**B3** (6, 4)	**C1** (1, 2)	**C2** (4, 9)

Distance: Euclidean

$$d(p, q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Initial centroids: A1 (2, 10), B1 (5, 8) and C1 (1, 2) are chosen as the centres of clusters 1, 2 and 3.



Red rings = initial centroids

Solved Example: Iteration 1

Find each point's distance to the 3 initial centroids, then assign it to the nearest one.

Point	(x, y)	Dist. to c1 (2, 10)	Dist. to c2 (5, 8)	Dist. to c3 (1, 2)	Cluster
A1	(2, 10)	**0.00**	3.61	8.06	**1**
A2	(2, 5)	5.00	4.24	**3.16**	**3**
A3	(8, 4)	8.49	**5.00**	7.28	**2**
B1	(5, 8)	3.61	**0.00**	7.21	**2**
B2	(7, 5)	7.07	**3.61**	6.71	**2**
B3	(6, 4)	7.21	**4.12**	5.39	**2**
C1	(1, 2)	8.06	7.21	**0.00**	**3**
C2	(4, 9)	2.24	**1.41**	7.62	**2**

Shaded cell = smallest distance → the point joins that cluster.

Current centroids

- c1 = (2, 10)
- c2 = (5, 8)
- c3 = (1, 2)

New centroids

- **c1 = (2, 10)**
  mean of A1
- **c2 = (6, 6)**
  mean of A3, B1, B2, B3, C2
- **c3 = (1.5, 3.5)**
  mean of A2, C1
  mean of (2,1), (5,2)

Solved Example: Iteration 2

Repeat with the updated centroids: C2 moves to a new cluster.

Point	(x, y)	Dist. to c1 (2, 10)	Dist. to c2 (6, 6)	Dist. to c3 (1.5, 3.5)	Cluster
A1	(2, 10)	**0.00**	5.66	6.52	1
A2	(2, 5)	5.00	4.12	**1.58**	3
A3	(8, 4)	8.49	**2.83**	6.52	2
B1	(5, 8)	3.61	**2.24**	5.70	2
B2	(7, 5)	7.07	**1.41**	5.70	2
B3	(6, 4)	7.21	**2.00**	4.53	2
C1	(1, 2)	8.06	6.40	**1.58**	3
C2	(4, 9)	**2.24**	3.61	6.04	1 *(was 2)*

Shaded cell = smallest distance → the point joins that cluster.

Current centroids

- c1 = (2, 10)
- c2 = (6, 6)
- c3 = (1.5, 3.5)

New centroids

- **c1 = (3, 9.5)**
  mean of A1, C2
- **c2 = (6.5, 5.25)**
  mean of A3, B1, B2, B3
- **c3 = (1.5, 3.5)**
  mean of A2, C1

Solved Example: Iteration 3

Repeat with the updated centroids: B1 moves to a new cluster.

Point	(x, y)	Dist. to c1 (3, 9.5)	Dist. to c2 (6.5, 5.25)	Dist. to c3 (1.5, 3.5)	Cluster
A1	(2, 10)	**1.12**	6.54	6.52	**1**
A2	(2, 5)	4.61	4.51	**1.58**	**3**
A3	(8, 4)	7.43	**1.95**	6.52	**2**
B1	(5, 8)	**2.50**	3.13	5.70	**1** *(was 2)*
B2	(7, 5)	6.02	**0.56**	5.70	**2**
B3	(6, 4)	6.26	**1.35**	4.53	**2**
C1	(1, 2)	7.76	6.39	**1.58**	**3**
C2	(4, 9)	**1.12**	4.51	6.04	**1**

Shaded cell = smallest distance → the point joins that cluster.

Current centroids

- c1 = (3, 9.5)
- c2 = (6.5, 5.25)
- c3 = (1.5, 3.5)

New centroids

- **c1 = (3.67, 9)**
  mean of A1, B1, C2
- **c2 = (7, 4.33)**
  mean of A3, B2, B3
- **c3 = (1.5, 3.5)**
  mean of A2, C1

Solved Example: Iteration 4

Repeat with the updated centroids: every point stays in the same cluster.

Point	(x, y)	Dist. to c1 (3.67, 9)	Dist. to c2 (7, 4.33)	Dist. to c3 (1.5, 3.5)	Cluster
A1	(2, 10)	1.94	7.56	6.52	1
A2	(2, 5)	4.33	5.04	1.58	3
A3	(8, 4)	6.62	1.05	6.52	2
B1	(5, 8)	1.67	4.18	5.70	1
B2	(7, 5)	5.21	0.67	5.70	2
B3	(6, 4)	5.52	1.05	4.53	2
C1	(1, 2)	7.49	6.44	1.58	3
C2	(4, 9)	0.33	5.55	6.04	1

Shaded cell = smallest distance → the point joins that cluster.

Current centroids

- c1 = (3.67, 9)
- c2 = (7, 4.33)
- c3 = (1.5, 3.5)

Converged ✓

No point changed cluster, so the centroids stay the same.

The algorithm stops here.

Solved Example: Final Clusters

The three clusters K-means found for our 8 points.

1

Cluster 1: A1, B1, C2

Final centroid c1 = (3.67, 9)

2

Cluster 2: A3, B2, B3

Final centroid c2 = (7, 4.33)

3

Cluster 3: A2, C1

Final centroid c3 = (1.5, 3.5)

Converged after **4 iterations** (no change in the last one).



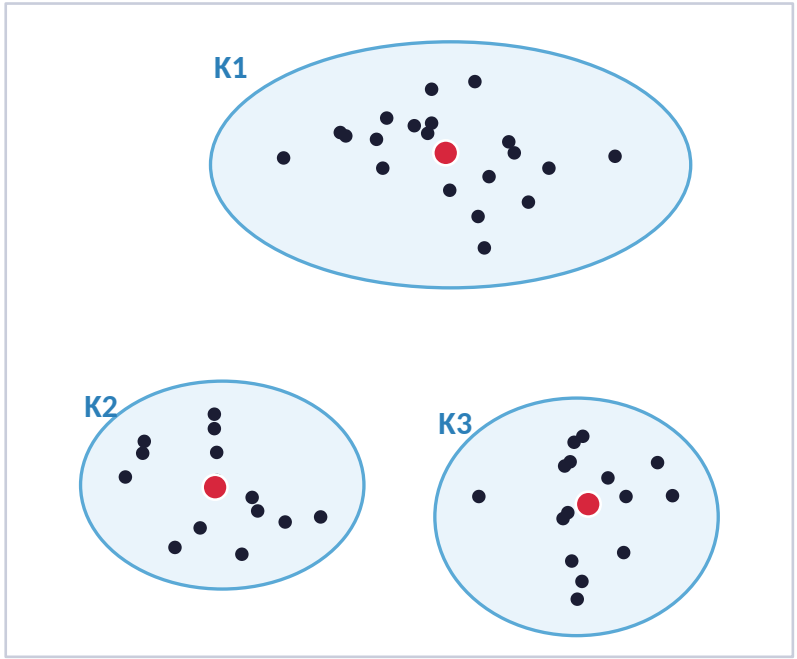
Diamonds = final centroids

The Challenge: Choosing k

- We need to select the initial centroids for each of the clusters.
- It requires us to specify the number of clusters (k) in advance.
- So, how do we select the optimal number of clusters (k) for the given data set?

There are two methods to select the optimal number of clusters in the K-Means algorithm:

1. **Elbow Method**
2. **Silhouette Method**



Here we can see 3 groups – but with real, high-dimensional data we usually cannot "see" k.

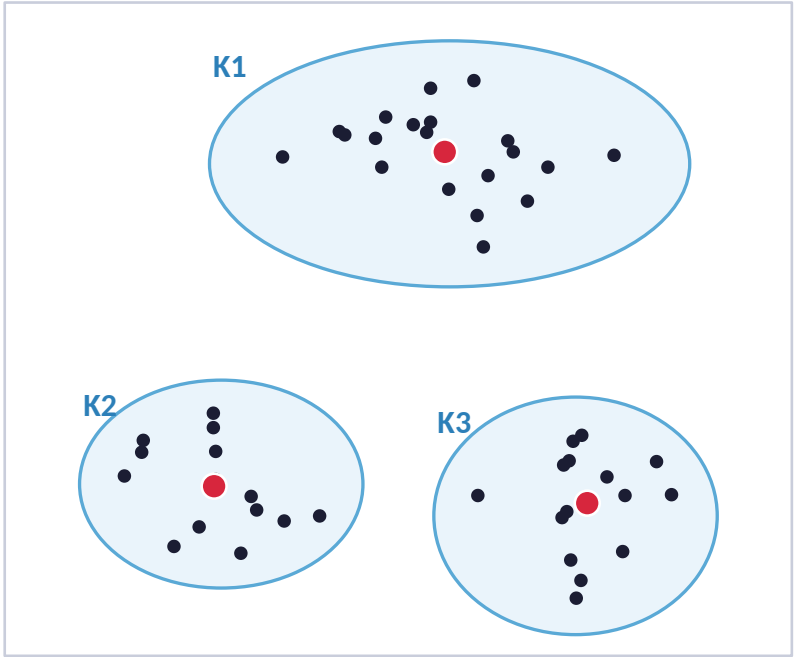
Elbow Method: Steps 1 and 2

1 **Step 1:** Apply the K-Means clustering algorithm and form the clusters for different values of k. For example, k = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10].

2 **Step 2:** For each k, calculate the within-cluster sum of squares (WCSS).

$$\text{WCSS} = \sum_{C_k}^{C_n} \left(\sum_{d_i \text{ in } C_i}^{d_m} distance(d_i, C_k)^2 \right)$$

Where C is the cluster centroids and d is the data point in each cluster.
In words: add up the squared distance from every point to its own centroid. Small WCSS = tight clusters.



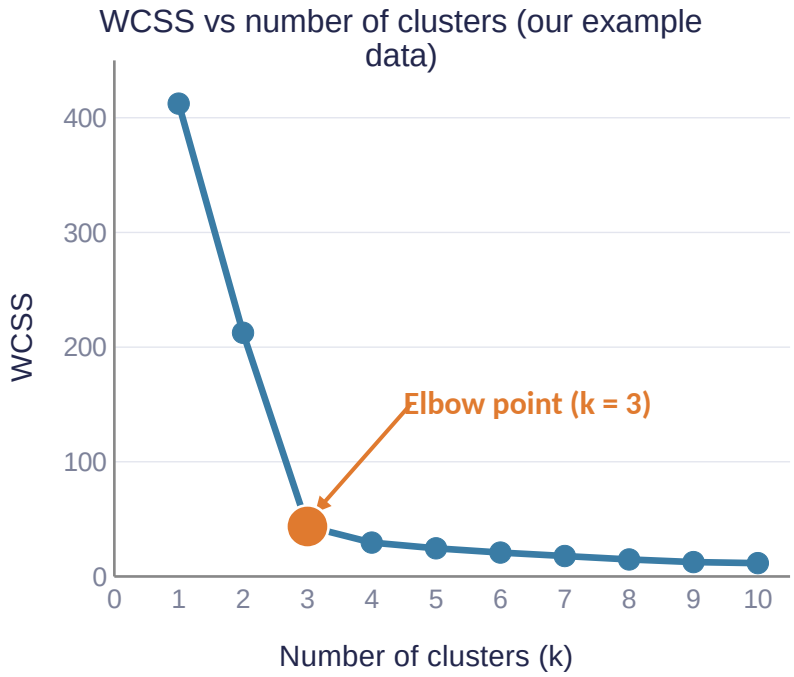
Example: k = 3 gives clusters K1, K2, K3. The WCSS sums the squared distance of each point to its red centroid.

Elbow Method: Steps 3 and 4

3 **Step 3:** Plot the curve of WCSS against the number of clusters.

4 **Step 4:** The location of the bend (the "elbow") in the plot is generally considered an indicator of the approximate number of clusters.

Why the elbow? Adding clusters always lowers WCSS. Before the elbow, each extra cluster helps a lot. After it, the gain is small – extra clusters just split real groups. In our data the elbow is at **k = 3**.



Advantages and Disadvantages

Advantages of K-Means

1. K-means algorithm is simple, easy to understand, and easy to implement.
2. It is also efficient: the time taken to cluster rises linearly with the number of data points.
3. No other clustering algorithm performs better than K-means.*

Disadvantages of K-Means

1. The user needs to specify an initial value of K.
2. The process of finding the clusters may not converge.*
3. Need to select the initial centroids for each of the clusters.

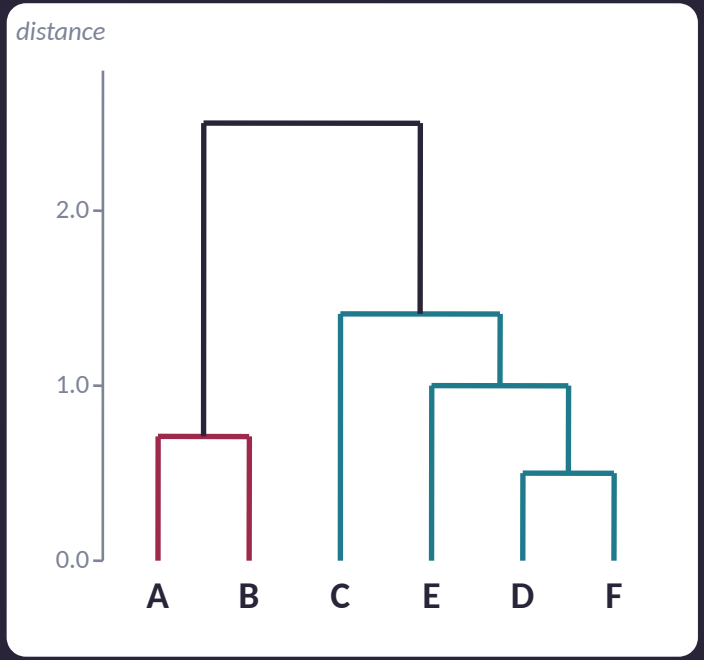
** **Teaching notes:** K-means is very popular, but whether it is "best" depends on the data – other methods (e.g. DBSCAN, hierarchical) do better for odd-shaped clusters. In practice K-means always stops, but it may settle on a poor (local) solution, so it is usually run several times with different starting centroids.*

Key Takeaways

1. K-means is a distance-based, unsupervised algorithm that groups data into k clusters
2. Each cluster is represented by its centroid (the mean of its points)
3. Repeat: assign points to the nearest centroid, then recompute centroids
4. Stop when no point changes group (centroids stabilize)
5. Choose k with the Elbow method (bend in WCSS) or the Silhouette method

Hierarchical Clustering

Agglomerative and divisive methods, dendrograms and solved examples



Why Hierarchical Clustering Matters

Many real datasets have groups inside groups. Hierarchical clustering reveals this whole structure instead of a single flat split.

No need to fix k first

Build the tree once, then choose the number of clusters by cutting it at any level.

Shows nested structure

Reveals sub-groups inside groups, like species → genus → family.

Easy to visualise

The dendrogram shows which points join, in what order, and how far apart they are.

Deterministic

Same data and settings always give the same tree – no random starting centroids.

Flexible shapes

With single linkage it can follow long, non-spherical clusters that K-means misses.

Widely used

Gene expression analysis, taxonomy, customer segmentation, document organisation.

Hierarchical Clustering Algorithm

- Hierarchical clustering is a **connectivity-based unsupervised clustering model** that groups the data points together that are close to each other based on the **measure of similarity or distance**.

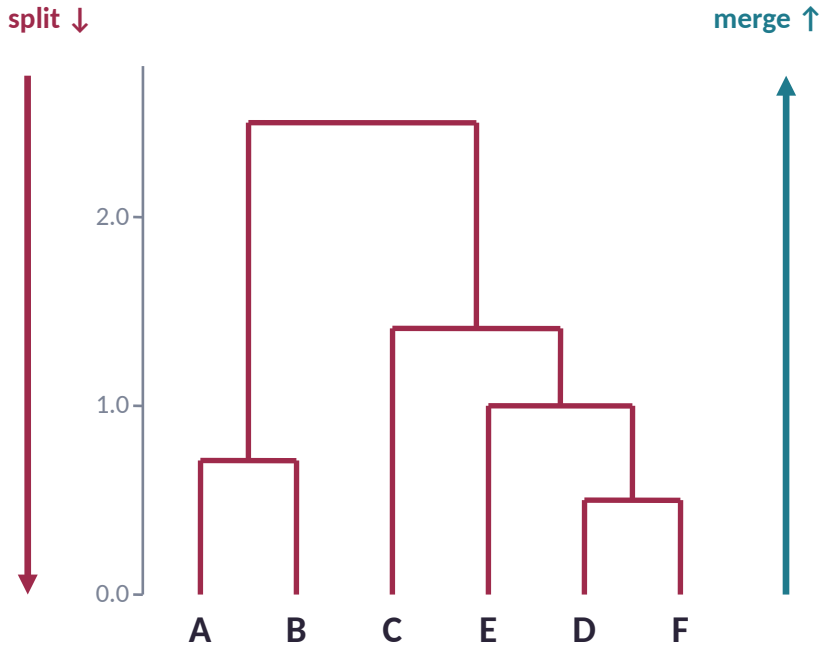
- The assumption is that **data points that are close to each other** are **more similar or related** than data points that are farther apart.

What Is Hierarchical Clustering?

- An unsupervised learning method that builds a hierarchy (tree) of clusters.
- Similar points are grouped step by step, based on the distance between them.
- The result is shown as a tree diagram called a dendrogram.
- Two approaches: Agglomerative (bottom-up: merge) and Divisive (top-down: split).

Agglomerative
bottom-up
Start: every point is its own cluster. Repeatedly MERGE the two closest clusters.

Divisive
top-down
Start: all points in one cluster. Repeatedly SPLIT a cluster into two.



Agglomerative goes up the tree (merge); divisive goes down (split).

Calculating Distance Between Points

Euclidean (most common) Straight-line distance

$$d(p, q) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Example: $d(A, B) = \sqrt{(0.5^2 + 0.5^2)} = \sqrt{0.5} = 0.71$

Manhattan Sum of absolute differences (city blocks)

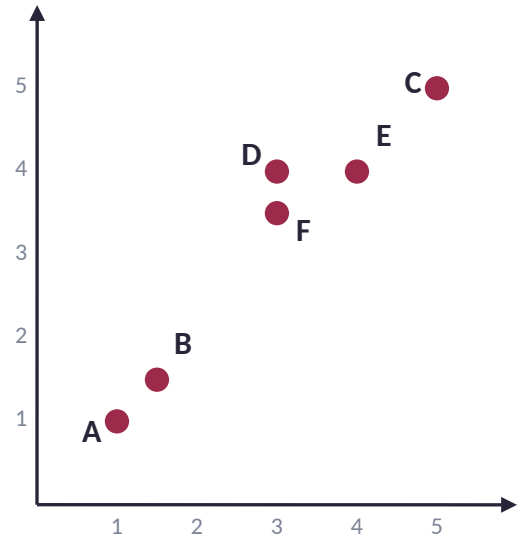
$$d(p, q) = |x_1 - x_2| + |y_1 - y_2|$$

Example: $d(A, B) = 0.5 + 0.5 = 1.00$

Others Chosen to suit the data

Minkowski (general form), Cosine (text), Hamming (categories)

Example: Same choice must be used throughout



Our example points: A(1, 1), B(1.5, 1.5), C(5, 5), D(3, 4), E(4, 4), F(3, 3.5)

Tip: scale features first, so one large-valued feature does not dominate the distance.

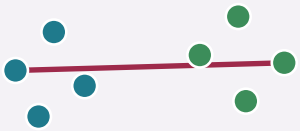
Distance Between Clusters: Linkage

Once points are grouped, we need a rule for the distance between two clusters. This rule is called the linkage.



Single

Distance of the closest pair of points: min d(a, b).



Complete

Distance of the farthest pair of points: max d(a, b).



Average

Average distance over all pairs of points.



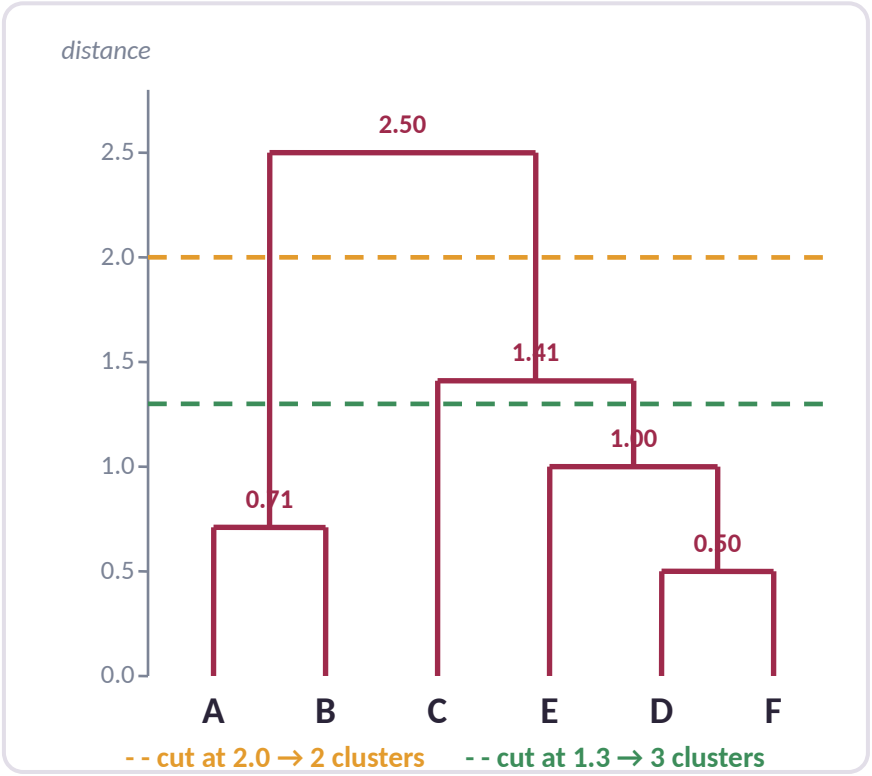
Centroid / Ward

Distance between cluster centres; Ward merges the pair that adds the least variance.

In our solved example we use single linkage. Different linkages can give different trees from the same data.

The Dendrogram

- A tree diagram that records every merge (or split).
- Leaves at the bottom are the individual data points.
- The height of a join = the distance at which those clusters were merged.
- Short joins = very similar points; tall joins = very different groups.
- Cut the tree with a horizontal line: the number of vertical lines it crosses = the number of clusters.
- A good cut is where the vertical gap between joins is largest.



Agglomerative Hierarchical Clustering Algorithm

- Agglomerative hierarchical clustering follows the **bottom-up approach**.
- Initially **each data point** is considered as a **singleton cluster** and then successively we **merge the data points** which are close to each other.
- The **process is repeated** until all clusters have been merged into a single cluster that contains all data.
- This clustering algorithm **does not require** us to prespecify the number of clusters.

Agglomerative Clustering: Steps

1. **Start**
   Treat each of the n data points as its own cluster.

2. **Distance matrix**
   Compute the distance between every pair of clusters (e.g. Euclidean).

3. **Merge**
   Find the two closest clusters and merge them into one.

4. **Update**
   Recompute the distances from the new cluster to all others using the linkage rule.

5. **Repeat**
   Repeat steps 3–4 until only one cluster is left (n − 1 merges).

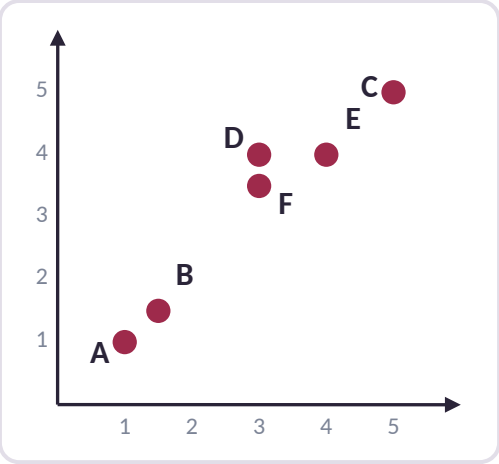
6. **Dendrogram**
   Draw the merges as a tree and cut it to get the desired number of clusters.

Single-linkage update rule: d(new cluster, X) = the smaller of the old distances. For example, d(DF, A) = min(d(D, A), d(F, A)).

Solved Example: Data and Distance Matrix

Six points, Euclidean distance, single linkage.

Point	x	y
A	1	1
B	1.5	1.5
C	5	5
D	3	4
E	4	4
F	3	3.5



	A	B	C	D	E	F
A	0					
B	0.71	0				
C	5.66	4.95	0			
D	3.61	2.92	2.24	0		
E	4.24	3.54	1.41	1.00	0	
F	3.20	2.50	2.50	**0.50**	1.12	0

How one entry is computed: $d(D, F) = \sqrt{((3 - 3)^2 + (4 - 3.5)^2)} = \sqrt{0.25} = 0.50$. **The smallest value in the matrix (highlighted) is d(D, F) = 0.50, so D and F merge first.**

Solved Example: Merges 2 and 3

Before merge 2: 5 clusters

	A	B	C	E	DF
A	0				
B	**0.71**	0			
C	5.66	4.95	0		
E	4.24	3.54	1.41	0	
DF	3.20	2.50	2.24	1.00	0

After merging D & F, update with min: d(DF, A) = min(3.61, 3.20) = 3.20; d(DF, E) = min(1.00, 1.12) = 1.00; d(DF, C) = min(2.24, 2.50) = 2.24.
Smallest: d(A, B) = 0.71 → merge A and B.

Before merge 3: 4 clusters

	C	E	DF	AB
C	0			
E	1.41	0		
DF	2.24	**1.00**	0	
AB	4.95	3.54	2.50	0

Update: d(AB, DF) = min(3.20, 2.50) = 2.50; d(AB, E) = min(4.24, 3.54) = 3.54; d(AB, C) = min(5.66, 4.95) = 4.95.
Smallest: d(E, DF) = 1.00 → merge E with DF.

Solved Example: Merges 4 and 5

Before merge 4: 3 clusters

	C	AB	EDF
C	0		
AB	4.95	0	
EDF	**1.41**	2.50	0

Before merge 5: 2 clusters

	AB	CEDF
AB	0	
CEDF	**2.50**	0

Update: d(EDF, C) = min(1.41, 2.24) = 1.41; d(EDF, AB) = min(3.54, 2.50) = 2.50.
Smallest: d(C, EDF) = 1.41 → merge C with EDF.

Only two clusters remain: AB and CEDF, at distance 2.50.
Final merge at 2.50 → one cluster containing all six points. Done after n − 1 = 5 merges.

Solved Example: The Dendrogram

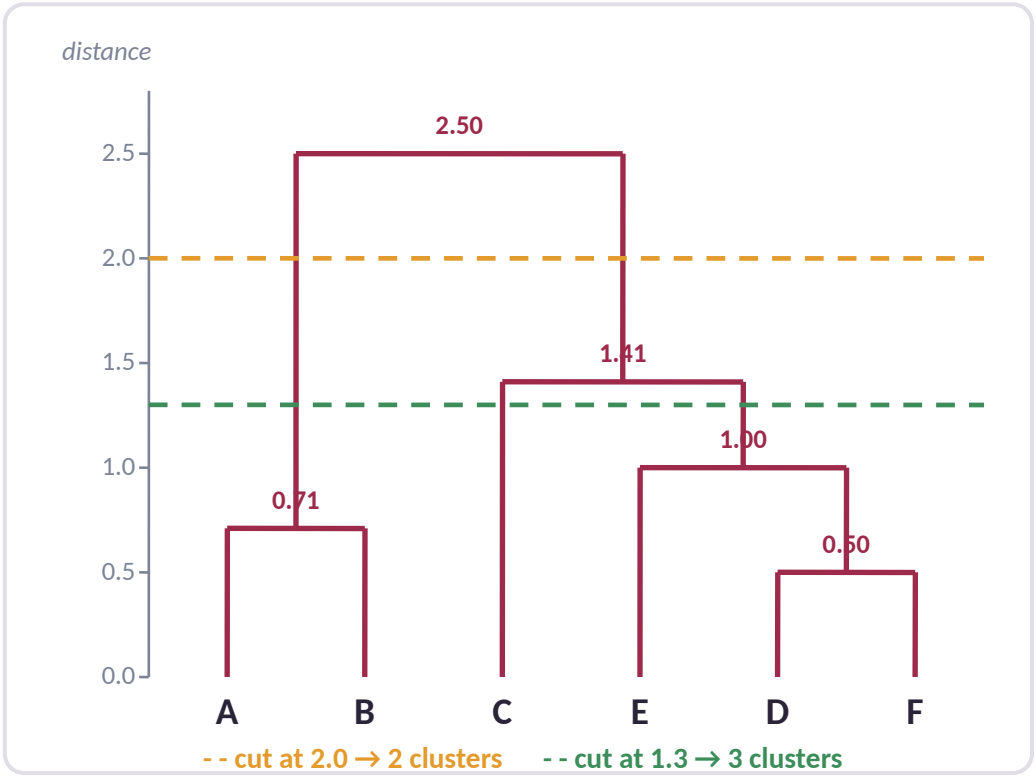
Step	Merge	Distance
1	D + F	0.50
2	A + B	0.71
3	E + DF	1.00
4	C + EDF	1.41
5	AB + CEDF	2.50

Reading the cut

Cut at 2.0 → 2 clusters: {A, B} and {C, D, E, F}.

Cut at 1.3 → 3 clusters: {A, B}, {C}, {D, E, F}.

The biggest gap (1.41 → 2.50) suggests k = 2.



Agglomerative Hierarchical Clustering Algorithm

- Given a dataset (d_1, d_2, d_3, ...d_N) of size N
- #compute the distance matrix
- for i=1 to N:
  - for j=1 to i:
  - dis_mat[i][j] = distance[d_i, d_j]
  - **# as the distance matrix is symmetric about the primary diagonal, so we compute only**
  - **# lower part of the primary diagonal**
- each data point is a singleton cluster
- **repeat**
  - merge the two-cluster having minimum distance
  - update the distance matrix

DIANA - DIvisive ANAlysis Clustering – Example

- Given the dataset {A, B, C, D, E} and the distance matrix in Table.
- Apply divisive analysis clustering algorithm (DIANA) to form clusters.

	A	B	C	D	E
A	0	1	2	2	3
B	1	0	2	4	3
C	2	2	0	1	5
D	2	4	1	0	3
E	3	3	5	3	0

DIANA - Divisive Analysis Clustering – Example

- **Step 1:** Initially $C_l = \{A, B, C, D, E\}$
- **Step 2:** $C_i = C_l$ and $C_j = \emptyset$
- **Step 3: Initial iteration**
- Let us calculate the average dissimilarities of the objects in C_i with the other objects in C_i.

	A	B	C	D	E
A	0	1	2	2	3
B	1	0	2	4	3
C	2	2	0	1	5
D	2	4	1	0	3
E	3	3	5	3	0

- **Average dissimilarity of A**

= 1/4 (d(a, b) + d(a, c) + d(a, d) + d(a, e)) = 1/4 (1 + 2 + 2 + 3) = 8/4 = **2.00**

Average dissimilarity of A

= 1/4 (d(a, b) + d(a, c) + d(a, d) + d(a, e)) = 1/4 (1 + 2 + 2 + 3) = 8/4 = **2.00**

- **Average dissimilarity of B**

= 1/4 (d(b, a) + d(b, c) + d(b, d) + d(b, e)) = 1/4 (1 + 2 + 4 + 3) = 10/4 = **2.50**

- **Average dissimilarity of C**

= 1/4 (d(c, a) + d(c, b) + d(c, d) + d(c, e)) = 1/4 (2 + 2 + 1 + 5) = 10/4 = **2.50**

- **Average dissimilarity of D**

= 1/4 (d(d, a) + d(d, b) + d(d, c) + d(d, e)) = 1/4 (2 + 4 + 1 + 3) = 10/4 = **2.50**

- **Average dissimilarity of E**

= 1/4 (d(e, a) + d(e, b) + d(e, c) + d(e, d)) = 1/4 (3 + 3 + 5 + 3) = 14/4 = **3.50**

DIANA - Divisive Analysis Clustering – Solved Example

A	B	C	D	E
2	2.5	2.5	2.5	**3.5**

	A	B	C	D	E
A	0	1	2	2	3
B	1	0	2	4	3
C	2	2	0	1	5
D	2	4	1	0	3
E	3	3	5	3	0

The highest average dissimilarity is **3.50**, for object **E**.

So, we move **E** to the New cluster C_j

C_i = {A, B, C, D} and C_j = {E}

DIANA - Divisive Analysis Clustering – Solved Example

- **Step 3: Remaining iterations** $C_i = \{A, B, C, D\}$ and $C_j = \{E\}$
- We calculate distance for each object $x \in C_i$:
- **D_x = avg dissimilarity of x with objects in C_i − dissimilarity of x with C_j**

	A	B	C	D	E
A	0	1	2	2	3
B	1	0	2	4	3
C	2	2	0	1	5
D	2	4	1	0	3
E	3	3	5	3	0

$D_a = 1/3\ (d(a, b) + d(a, c) + d(a, d)) - d(a, e)$

$= 1/3\ (1 + 2 + 2) - 3 = 5/3 - 3 = -4/3 \approx$ **−1.33**

$D_b = 1/3\ (d(b, a) + d(b, c) + d(b, d)) - d(b, e)$

$= 1/3\ (1 + 2 + 4) - 3 = 7/3 - 3 = -2/3 \approx$ **−0.67**

$D_c = 1/3\ (d(c, a) + d(c, b) + d(c, d)) - d(c, e)$

$= 1/3\ (2 + 2 + 1) - 5 = 5/3 - 5 = -10/3 \approx$ **−3.33**

$D_d = 1/3\ (d(d, a) + d(d, b) + d(d, c)) - d(d, e)$

$= 1/3\ (2 + 4 + 1) - 3 = 7/3 - 3 = -2/3 \approx$ **−0.67**

DIANA - Divisive Analysis Clustering – Solved Example

- $D_a = -1.33$
- $D_b = -0.67$
- $D_c = -3.33$
- $D_d = -0.67$

	A	B	C	D	E
A	0	1	2	2	3
B	1	0	2	4	3
C	2	2	0	1	5
D	2	4	1	0	3
E	3	3	5	3	0

All $D_x < 0$, so **no object is moved**.

Thus, we **finalize** this split as: **$C_i = \{A, B, C, D\}$ and $C_j = \{E\}$**

Now, we start splitting **$C_l = C_i = \{A, B, C, D\}$**

DIANA - Divisive Analysis Clustering – Solved Example

- **Step 1:** Initially $C_i = \{A, B, C, D\}$
- **Step 2:** $C_i = C_i$ and $C_j = \varnothing$
- **Step 3: Initial iteration**
- We start by computing the **average dissimilarity** of each object with the rest of the group.

	A	B	C	D	E
A	0	1	2	2	3
B	1	0	2	4	3
C	2	2	0	1	5
D	2	4	1	0	3
E	3	3	5	3	0

Average dissimilarity of **A**:

$= 1/3\ (d(a, b) + d(a, c) + d(a, d)) = 1/3\ (1 + 2 + 2) = 5/3 \approx$ **1.67**

DIANA - Divisive Analysis Clustering – Solved Example

- Average dissimilarity of **B**:

= 1/3 (d(b, a) + d(b, c) + d(b, d)) = 1/3 (1 + 2 + 4) = 7/3 ≈ **2.33**

- Average dissimilarity of **C**:

= 1/3 (d(c, a) + d(c, b) + d(c, d)) = 1/3 (2 + 2 + 1) = 5/3 ≈ **1.67**

- Average dissimilarity of **D**:

= 1/3 (d(d, a) + d(d, b) + d(d, c)) = 1/3 (2 + 4 + 1) = 7/3 ≈ **2.33**

DIANA - Divisive Analysis Clustering – Solved Example

- $D_a = 1.67$
- $D_b = 2.33$
- $D_c = 1.67$
- $D_d = 2.33$

	A	B	C	D	E
A	0	1	2	2	3
B	1	0	2	4	3
C	2	2	0	1	5
D	2	4	1	0	3
E	3	3	5	3	0

- The highest average dissimilarity is **2.33**, and both **B** and **D** have this value.
- Let us **arbitrarily choose B** and put in to c_j.
- $C_i = \{A, C, D\}$ and $C_j = \{B\}$

DIANA - Divisive Analysis Clustering – Solved Example

- **Step 3: Remaining iterations** $C_i = \{A, C, D\}$ and $C_j = \{B\}$
- We calculate distance for each object $x \in C_i$:
- **D_x = avg dissimilarity of x with objects in C_i − dissimilarity of x with C_j**

$D_a = 1/2\ (d(a, c) + d(a, d)) - 1/1\ (d(a, b))$

$= 1/2\ (2 + 2) - 1 = 4/2 - 1 = 2 - 1 =$ **1.0**

$D_c = 1/2\ (d(c, a) + d(c, d)) - 1/1\ (d(c, b))$

$= 1/2\ (2 + 1) - 2 = 3/2 - 2 = 1.5 - 2 =$ **−0.5**

$D_d = 1/2\ (d(d, a) + d(d, c)) - 1/1\ (d(d, b))$

$= 1/2\ (2 + 1) - 4 = 3/2 - 4 = 1.5 - 4 =$ **−2.5**

	A	B	C	D	E
A	0	1	2	2	3
B	1	0	2	4	3
C	2	2	0	1	5
D	2	4	1	0	3
E	3	3	5	3	0

DIANA - Divisive Analysis Clustering – Solved Example

- $D_a = 1$
- $D_c = -0.5$
- $D_d = -2.5$

	A	B	C	D	E
A	0	1	2	2	3
B	1	0	2	4	3
C	2	2	0	1	5
D	2	4	1	0	3
E	3	3	5	3	0

Highest average dissimilarity is **1**, for object **A**.

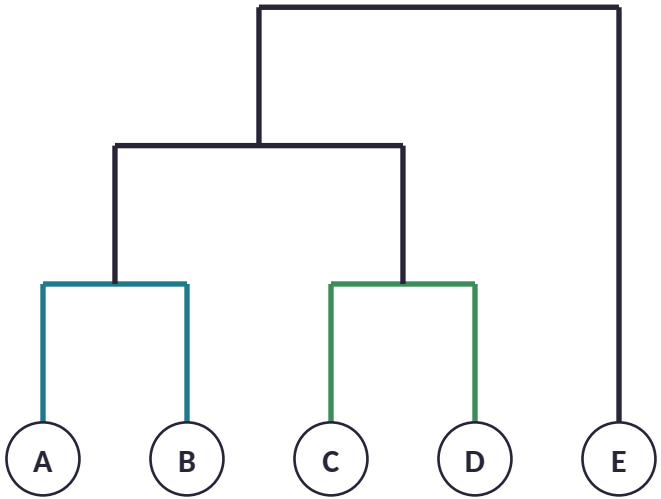
So, we move **A** to the New cluster C_j

C_i = {C, D} and C_j = {B, A}

DIANA - Divisive Analysis Clustering – Solved Example

- Final Clusters
- **{B, A}, {C, D} and {E}**

- **Dendrogram**



	A	B	C	D	E
A	0	1	2	2	3
B	1	0	2	4	3
C	2	2	0	1	5
D	2	4	1	0	3
E	3	3	5	3	0

Agglomerative vs Divisive

	Agglomerative	Divisive
Direction	Bottom-up: merge	Top-down: split
Starts with	n single-point clusters	One cluster with all points
Each step	Merge the two closest clusters	Split one cluster into two
Good at	Finding small, local groups	Finding the big, top-level groups
Cost	Cheaper, most widely used	More expensive (many ways to split)
Example algorithm	AGNES	DIANA

Hierarchical vs K-Means Clustering

Aspect	Hierarchical	K-Means
Number of clusters	Not needed in advance – cut the dendrogram	Must choose k first (elbow / silhouette)
Output	Tree of nested clusters (dendrogram)	One flat set of k clusters
Approach	Merge (or split) step by step	Assign to nearest centroid, update, repeat
Result	Deterministic: same tree every run	Depends on the random starting centroids
Cluster shape	Depends on linkage; single can follow long shapes	Best for round (spherical), similar-sized clusters
Speed / data size	Slow, $O(n^2)$ memory: small–medium data	Fast, scales to very large data
Changing decisions	A merge or split can never be undone	Points can move between clusters each round

Rule of thumb: hierarchical for small data or when you want to see the structure; K-means for large data when k is roughly known.